

Reading APFS
Notes Toward a Fast Index of an Apple File System Volume

apfs-fastindex project

2026-05-21

Contents

Preface	v
I The Problem	1
1 Why APFS Resists MFT-Style Indexing	2
1.1 What APFS Provides	2
1.2 The Right Question	2
1.3 The Defended Core	3
2 Method	4
2.1 Paired Probes	4
2.2 The Smallest Useful Fixture	4
2.3 The Style Of A Finding	5
II The Container and the Object Map	6
3 The Container	7
3.1 Block Zero As Locator	7
3.2 Selecting The Checkpoint	7
3.2.1 Why The Highest Valid XID	8
3.2.2 Live State Is Not Pinned	8
3.3 The Checkpoint Map Ring	9
3.4 Roots Reachable From The Selected Checkpoint	9
4 The Object Map	10
4.1 Lookup By (OID, Max-XID)	10
4.2 Hard Stops At OMAP Open	10
4.3 Hard Stops At OMAP Lookup	11
4.4 Empirical Correction: B-Tree TOC Layout (EX-12)	11
4.5 Cross-Tool Validation	11
4.6 What The OMAP Does Not Do	12
III The File-System Tree	13
5 The File-System Tree	14

5.1	The Rule	14
5.2	The Probe (EX-14, EX-15)	14
5.3	Object-Header Validation	15
5.4	Required Record Families	15
5.5	Topology Summary	16
6	Record Bodies and the Xfield Cursor	17
6.1	The Cursor Rule	17
6.2	Where The Reference Reads Otherwise	18
6.3	Evidence From The Proof Fixture	18
6.4	The Required Xfield Types	19
6.5	Field-Level Cross-Tool Equality	19
7	The Fail-Closed Boundary	21
7.1	The Two Chokepoints	21
7.2	The Per-Record Catalogue	21
7.2.1	Short bodies	22
7.2.2	Malformed names	22
7.2.3	Malformed xfields	22
7.2.4	Malformed xattrs	22
7.3	The Cross-Record Cases	23
7.4	Why The Catalogue Is Worth Its Weight	23
7.5	Object-Header Storage Classes	23
IV	Names and Sizes	25
8	Sizes and Their Precedences	26
8.1	Logical Size	26
8.1.1	The Precedence Rule	26
8.1.2	The Load-Bearing Case: Compression	28
8.1.3	Sparse Bytes Is A Hint, Not A Size	28
8.1.4	Per-Inode Results	28
8.2	Allocated Size	29
8.2.1	The Precedence Rule	29
8.2.2	Why <code>allocated_size</code> Is The Only Candidate	29
8.2.3	The Sparse Identity (EX-22, EX-26)	30
8.2.4	The <code>decmpfs</code> Rule (EX-26)	30
8.2.5	Per-Inode Results	31
8.3	Real (Clone-Deduplicated) Size	31
8.3.1	The Formula (EX-27)	31
8.3.2	Storage Class Of The Extent-Reference Tree	32
8.3.3	Fallback Behaviour	32
8.4	Directory Aggregates	32
8.5	Out Of Scope	33

9	Names Across Volume Modes	34
9.1	Verbatim Preservation	34
9.2	Two Volume Modes Exist	34
9.3	What The Modes Affect	34
9.3.1	The Lookup Oracle	35
9.3.2	Rust vs POSIX Equality	35
9.4	Lookup-by-Name Is Not Claimed	35
9.5	Names and Aggregates	35
9.6	What A Future Lookup-by-Name Would Need	36
V	Boundaries	37
10	Identity, Reuse, and Cache Keys	38
10.1	Why OID Alone Is Unsafe	38
10.2	The Shape Of A Safe Cache Key	38
10.3	What EX-07 Proved	39
10.4	The Shipping CLI Cache (EX-30)	39
10.5	What R4 v2 Needs	40
10.6	What This Chapter Forecloses	40
11	Support, Readability, and the Fallback	42
11.1	Four Properties	42
11.2	The Current Allowlist	42
11.3	Live-Volume Raw Mode (EX-28)	43
11.4	Local-Snapshot Enumeration (EX-29)	44
11.5	Fail-Closed Triggers	44
11.6	The Fallback	45
11.7	Resilience In The Fallback	45
11.8	Parallel Fallback	45
11.9	Firmlink Overlay Dedup	46
11.10	Auto-Dispatch	47
11.11	Why The Boundary Is Permanent	47
VI	Engineering	48
12	Measurement	49
12.1	The Standing Baseline	49
12.2	The Micro-Optimisation Pass	50
12.3	The Performance-Lever Map	52
12.3.1	The Parallel Slice In Product	53
12.4	The Structural-Density Pass	54
12.5	The Production-Shape Baseline (EX-30)	55
12.6	What The Numbers Do Not Say	56

13 Engineering Surface and Open Questions	57
13.1 The Native Shell	57
13.1.1 The FFI Boundary	57
13.1.2 Division Of Labour	58
13.1.3 Treemap Layout	58
13.1.4 Substring Search (EX-33)	58
13.1.5 What Is Not In The Shell	59
13.2 Boot-Root Semantics (EX-31)	59
13.3 Encryption (EX-32 Scratchpad)	60
13.4 Open Questions	60
13.4.1 Exclusive and Shared Bytes	60
13.4.2 Snapshot-Assisted Scanning	60
13.4.3 Persistent Incremental Cache	61
13.4.4 Encrypted Surfaces	61
13.4.5 Live Boot Disk Raw Mode	61
Glossary	62
Appendix: Experiment Register	67

Preface

This manual records what the `apfs-fastindex` project has learned about the Apple File System on disk, and about the problem of enumerating a single APFS volume quickly and correctly. It is written for a careful engineer who wants to read APFS metadata directly. Familiarity with copy-on-write filesystems and B-trees is assumed; APFS-specific knowledge is not.

Claim Labels

Three labels appear throughout the text and are never blurred.

Spec. Backed by Apple’s public reference¹ or by another durable specification.

Observation. Backed by a controlled experiment in this project, or by behaviour that converges across multiple independent open-source readers (`linux-apfs-rw`, `apfs-fuse`, `dissect.apfs`, `go-apfs`, `libfsapfs`, the Sleuth Kit’s APFS module).

Hypothesis. Plausible, useful for design, not yet proven. A hypothesis does not justify product behaviour; the parser does not emit a row that depends only on a hypothesis-class claim.

Where Spec and Observation disagree, Observation wins. The reference describes intent; the running system writes what is on disk, and the two occasionally disagree.

Oracles

Each feature owns its own oracle.

Namespace. POSIX traversal of the same volume at the same selected state.

Logical size. `st_size` per inode; for symlinks, the target byte length returned by `readlink`.

Allocated size. `st_blocks * 512`.

FS-record body. Field-equality between independent decoders against the same raw container.

Real (clone-deduplicated) size. $\sum \text{phys_ext.length} \cdot \text{block_size}$ from the on-disk extent-reference tree (no public per-file macOS oracle exists; `du -A` reports apparent size and does not dedup clones).

Incremental correctness. A fresh full reparse of the same selected state.

Every result names the oracle that verified it.

¹Apple File System Reference, retrieved 2026-05-13, <https://developer.apple.com/support/downloads/Apple-File-System-Reference.pdf>.

Fail-Closed Discipline

Outside the tested allowlist the parser stops with a typed error and the caller picks a fallback. Plausible-but-unproven is treated as “do not emit.” The cost of a silently wrong row is unbounded: it propagates into directory aggregates, into the treemap, and into decisions the user takes from the treemap. The cost of refusing a row is bounded: the fallback path produces a correct answer through supported APIs.

Scope

Each chapter takes one face of APFS, states what is known, presents the evidence, and notes what remains open. Where the implementation makes a non-obvious choice the choice is explained from first principles. What is not here: a tour of the repository, a list of files, a chronology of commits. Those live in the repository; the manual cites them only when citation supports a claim.

Part I

The Problem

Chapter 1

Why APFS Resists MFT-Style Indexing

WizTree’s speed on NTFS comes from the Master File Table: a single flat structure in which every file is one record. Indexing reduces to a sequential scan. No recursion, no per-file `stat`, no path traversal.

APFS provides no equivalent. There is no flat metadata table, by design.

1.1 What APFS Provides

An APFS container is a copy-on-write region of transactional B-trees. To enumerate one volume, a reader must traverse at least three of those trees under one pinned transaction state:

- a stable block-zero locator that points at the *checkpoint descriptor area*,
- the descriptor area, a ring of container superblocks (one per transaction), each a self-consistent view at that transaction,
- the *container object map* (OMAP), a B-tree resolving virtual object identifiers to physical block addresses under a chosen transaction,
- one or more *volume superblocks*, reached through the container OMAP,
- per-volume OMAPs, resolving the volume’s virtual objects,
- per-volume *file-system trees*, B-trees of variable-length records keyed by `(object_id, type)` carrying directory entries, inodes, extended attributes, hard-link records, and data-stream metadata.

Every metadata read therefore costs three indirections: container, volume, FS tree. Each indirection requires the correct OMAP and the correct transaction context.

1.2 The Right Question

What is the minimum set of structures we must traverse, in what order, under what state pin, to correctly enumerate one volume?

The answer fills most of this manual. In compressed form:

1. Locate the descriptor area from block zero.
2. Select one checkpoint from that area — the highest-XID checksum-valid container superblock.
3. Walk the checkpoint map; resolve the container OMAP.
4. Look up the target volume superblock through the container OMAP.
5. Open the volume OMAP; look up the FS-tree root.
6. Walk the FS tree, resolving internal-node child references through the volume OMAP at every level.
7. Decode FS-tree leaf records into directory entries, inodes, xattrs, sibling links, and dstreams.
8. Reconstruct paths, file identity, and size from those records.

Each step is independently load-bearing. Skip the OMAP lookups at the FS-tree internal level (chapter 5) and the walker reads random unallocated blocks. Pick the wrong checkpoint (chapter 3) and the walker reads an incoherent transaction. Mis-decode an xfield blob (chapter 6) and the parser silently emits wrong sizes.

1.3 The Defended Core

The manual’s scope is deliberately narrow:

- one APFS volume,
- at one pinned transaction state,
- correct namespace,
- per-file logical, allocated, and clone-deduplicated (“real”) size,
- fail closed outside the tested allowlist.

The scope is narrow because the cost of being approximately right is unbounded: aggregates derived from silently wrong rows propagate into treemaps, into “where did my disk space go” decisions, and into filesystem operations the user takes based on those numbers. The cost of being narrowly right and failing closed everywhere else is bounded: the fallback path produces a correct answer through supported APIs, more slowly.

Chapter 2

Method

This chapter records the probe-construction discipline the rest of the manual presupposes. It is short by design.

2.1 Paired Probes

The most informative experiments build the fixture, capture every oracle, run the candidate parser, and compare — all in one execution of one script. This rules out the drift modes that otherwise break a probe: a fixture rebuilt with slightly different operations, a volume remounted between captures, a parser pointed at a different container than the oracle.

Two patterns recur.

Detach-and-reattach. Build the fixture mounted; capture the mounted POSIX oracle; `hdiutil detach`; `hdiutil attach -nomount -readonly`; run the raw parser; diff. The detach fixes the visible checkpoint, so the raw parser and the mounted oracle agree on what state to compare.

Cross-tool agreement. Run the candidate parser and an independent third-party reader against the same raw container in the same script. Record the parser's resolved (`oid`, `paddr`, `xid`) samples; replay the object-header validation rules at every resolved `paddr`; confirm the two tools agree on the FS-tree root OID. A divergence in *which* checkpoint each tool selected is recorded as a caveat, not silenced.

2.2 The Smallest Useful Fixture

Probes prefer the smallest fixture that exercises the question. The proof fixture used through most of this manual contains two directories, a renamed file, a moved file, a hard link, a sparse 1 MiB file, a clone, an append after the clone, and a symlink: roughly seven user-visible entries.

That fixture is enough to surface:

- the FS-tree internal-OID-vs-paddr distinction (when the tree grows past one leaf node; see chapter 5),
- the xfield cursor rule (the sparse inode has three xfields, one of which has a non-multiple-of-eight `x_size`),
- logical-size precedence over ordinary, sparse, clone, hard-linked, and symlink rows,

- the unique-inode aggregate policy (hard links must not double-count).

Larger fixtures appear only when a specific question requires something the small fixture cannot express: the case-insensitive versus case-sensitive comparison of chapter 9, the sparse/decmpfs corpora of chapter 8, the clone-dedup fixture of chapter 8's “real-size” section.

2.3 The Style Of A Finding

Each finding is presented the same way: a one-line statement, the evidence, the implementation consequence, and the limits of what was checked. A claim must survive standing alone, and the proof must be present.

Part II

The Container and the Object Map

Chapter 3

The Container

Three rules govern reading an APFS container: block zero locates, it does not authorise; checkpoint selection picks the highest checksum-valid candidate; and the selected state must not advance during the scan. This chapter develops each rule.

3.1 Block Zero As Locator

APFS is copy-on-write at the container level. Each transaction writes a new container superblock (`nx_superblock_t`, henceforth NXSB) into the checkpoint descriptor area. Block zero is the stable known location at which a reader can find the descriptor area. It is updated lazily; the authoritative live NXSB is one of the copies in the descriptor ring.

The pieces of block zero a reader needs:

- `block_size` at offset `0x24`: validated to be a power of two within `[4096, 65536]`.
- `nx_xp_desc_blocks` at offset `0x68`: the length of the descriptor area in blocks.
- `nx_xp_desc_base` at offset `0x70`: the first block of the descriptor area. The high bit signals a *non-contiguous* layout, which this work does not support and which triggers a typed hard stop. A non-contiguous descriptor area would require a separate checkpoint-mapping-tree implementation.

Block zero is itself an APFS object and must validate as one: NXSB magic at offset `0x20`, object type `NX_SUPERBLOCK`, object id equal to the constant NX-superblock OID (1), and a valid Fletcher-64 checksum. A block zero that does not satisfy these is not an APFS container.

Observation. The `block_size` and descriptor base in block zero must agree with the selected NXSB's view of the same fields. A discrepancy means block zero is stale relative to a descriptor write that did not (yet) propagate. The parser rejects the source rather than guessing which view is correct.

3.2 Selecting The Checkpoint

The descriptor area is a contiguous run of `nx_xp_desc_blocks` blocks. Some are checkpoint maps (chapter 3.3); some are NXSB candidates. The selection rule:

1. Iterate every block in the descriptor range.

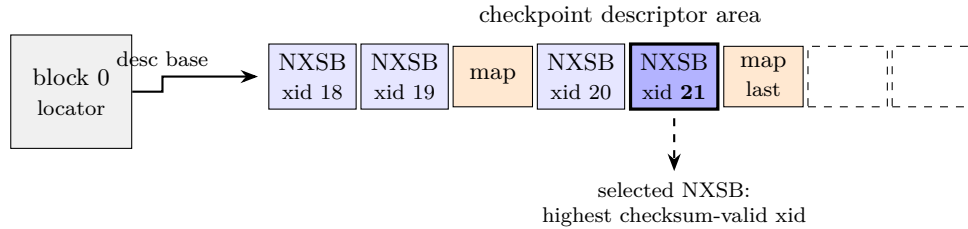


Figure 3.1: Block zero locates the descriptor ring; the selected NXSB is the highest checksum-valid candidate in the ring. Several earlier NXSBs may remain valid; **map** blocks carry checkpoint-map contents; trailing slots may be unwritten between commits.

2. For each, check the NXSB magic at offset 0x20.
3. If the magic matches, validate object type (NX_SUPERBLOCK) and Fletcher-64 checksum.
4. Record the candidate.
5. After the scan, pick the candidate with the highest XID.

The highest valid XID is the selected NXSB. Its XID becomes the *scan XID*, and every subsequent object resolution for the run is performed at this XID.

Blocks that carry NXSB magic but fail object-type or checksum validation are recorded as skipped descriptors with the reason. They are not silently ignored; a future support broadening must justify why they were skipped.

Observation. Three or four NXSB candidates is typical for a recently written container. The first execution of EX-15 against the EX-14 fixture saw four candidates with XIDs 18–21; all four passed strict validation. Multiple candidates is not a sign of corruption. Zero candidates is.

3.2.1 Why The Highest Valid XID

The candidates are not chronologically ordered in the descriptor area; they are written into a ring. The XID is the only monotonic identity for transaction order. The highest checksum-valid XID is the most recent transaction the writer completed. A higher XID with an invalid checksum is a transaction the writer started but did not finish; the parser treats those as failed and ignores them, with the result that selection is always self-consistent.

The rule has a corollary: a parser that picks a lower XID when the highest is valid has chosen a stale state for no reason and will disagree with anything else that picks the latest.

3.2.2 Live State Is Not Pinned

Observation. On a mounted APFS volume that other processes are using, the highest valid XID advances during the scan. Two reads of “latest” may return different transactions; mixing objects across transactions produces an incoherent namespace. EX-01 demonstrated this on a mounted lab image under write churn.

Raw mode therefore only runs on sources where state can be pinned: a detached image (the `.dmg` layer fixes the visible state at detach time), or a caller-supplied raw device under an explicit commitment that the checkpoint will not advance. Live mounted volumes are readable but not supported; chapter 11 develops the vocabulary.

3.3 The Checkpoint Map Ring

A selected NXSB references a chain of checkpoint maps (`checkpoint_map_phys_t`) in the descriptor area. The map chain carries the ephemeral object identifier table for the selected checkpoint: the OIDs and physical addresses of objects that live in memory between transactions and are written into the checkpoint data area at commit. The reader needs the map to resolve those ephemerals.

The walk rule:

1. Start at `nx_xp_desc_index` in the selected NXSB.
2. Read the block; validate as an APFS object with type `OBJECT_TYPE_CHECKPOINT_MAP`.
3. Record the (`oid`, `paddr`) mappings.
4. Advance through subsequent map blocks in the descriptor area.
5. Stop when a block carries the `CHECKPOINT_MAP_LAST` flag.

A chain that runs out of length before the last flag is malformed. The parser fails closed.

Empirical correction (EX-11). `OBJECT_TYPE_CHECKPOINT_MAP` blocks carry the `OBJ_PHYSICAL` storage flag, not `OBJ_EPHEMERAL`, despite holding the ephemeral-object lookup table. The Apple reference reads naturally as “the checkpoint-map block itself is ephemeral” — it is not. The validator expects `Physical` for these blocks. A synthetic malformed-map regression test covers the constant.

3.4 Roots Reachable From The Selected Checkpoint

The selected NXSB names five things the reader needs:

1. `nx_omap_oid`: the OID of the container OMAP.
2. `nx_fs_oid[]`: an array of volume superblock OIDs.
3. Feature, incompatible-feature, and read-only-compatible feature masks.
4. Reaper and space-manager OIDs (not used by an indexer; their presence is required, their content is not).
5. Container UUID (used for source identity in caches and diagnostics).

The incompatible-feature mask is the gate for format drift (chapter 11). Any bit the reader does not recognise stops the run.

Block zero, the descriptor area, and the checkpoint map together establish a single coherent transaction context. The OMAP machinery of the next chapter operates inside that context.

Chapter 4

The Object Map

APFS resolves virtual object identifiers to physical block addresses through a B-tree called an object map (OMAP). There is one per container (resolving volume superblocks and ephemerals) and one per volume (resolving the FS-tree root and its descendants). The OMAP is the mechanism by which a copy-on-write filesystem maintains stable object identity across mutations.

The lookup takes *two* keys, and the second is non-negotiable.

4.1 Lookup By (OID, Max-XID)

Spec. An OMAP key is (oid, xid). An OMAP value names a physical address, a size, and a flag word. The OMAP can hold multiple entries for a single OID, each at a different XID, corresponding to different versions of the object over time.

Spec. An OMAP lookup specifies an oid and a *max XID*. The B-tree returns the entry with the highest stored XID that is *less than or equal to* the requested max XID. The semantics are lower-bound rather than exact match: the lookup is asking "give me the version of oid that was current at or before this scan."

The resolver input is a triple: the OMAP context (which OMAP owns the OID), the OID, and the scan XID. The OID by itself does not determine which version of the object is meant; the scan XID does.

The scan XID is fixed for the entire run — the one selected from the descriptor area in chapter 3. Mixing scan XIDs across resolutions reintroduces the incoherence that pinning the checkpoint was supposed to prevent.

4.2 Hard Stops At OMAP Open

When the parser opens an OMAP — reads the `omap_phys_t` root object and validates it — the `omap_phys.flags` field gates whether the OMAP can be used at all.

OMAP_PHYS_ENCRYPTING.

OMAP_PHYS_DECRYPTING.

OMAP_PHYS_KEYROLLING.

OMAP_PHYS_CRYPTO_GENERATION_FLAG.

Any of these set means an encryption operation is in flight on the container or the volume. Raw mode cannot be supported in such a state: the bytes the reader sees do not yet match the bytes the running system will commit. The parser fails closed; the caller falls back to a supported API.

Observation. Any *unknown* bit in `omap_phys.flags` is also a hard stop. APFS reserves bits over time; an unknown bit means the running OS knows something the parser does not, and the parser must not infer behaviour. This is the format-drift discipline of chapter 11 applied at the smallest object that hosts it.

4.3 Hard Stops At OMAP Lookup

Each value entry returned by an OMAP lookup carries its own flag word. The flags decide whether the value is usable.

OMAP_VAL_DELETED. Negative result, not an error. The OID had a mapping at some earlier XID but is no longer visible at the scan XID. The caller interprets this as "the object is not present" and proceeds accordingly. The parser does not hard-stop.

OMAP_VAL_ENCRYPTED. The value is per-object encrypted. Raw mode cannot decrypt; hard stop.

OMAP_VAL_NOHEADER. The target block does not carry an `obj_phys_t` header at all. A structural assumption the rest of the parser depends on has been broken; hard stop.

OMAP_VAL_CRYPTO_GENERATION. Encryption-generation mismatch between the OMAP entry and the volume; hard stop.

unknown bits. Format drift; hard stop.

The discipline is tighter at lookup time than at open time only because lookup is also where the parser sees per-entry encryption behaviour. The principle is the same: known flag means known behaviour; unknown flag means the parser does not understand the running system and refuses to guess.

4.4 Empirical Correction: B-Tree TOC Layout (EX-12)

Observation. The B-tree leaf TOC `v_off` field measures the distance from the *end* of the value area to the *start* of the value. The value runs forward from that start for `fixed_value_size` bytes (or `v_len` for variable-length values).

The reference admits a reading under which the value extends *backward* from `v_off`. That reading produces nonsense OMAP values — `flags=0x10ffff` on the proof fixture — and corrupts every downstream lookup. The forward reading produces clean values (`flags=0x0`, `size=4096`, `paddr=...`) and matches every converged reader. A synthetic OMAP regression test catches any reintroduction of the bug.

4.5 Cross-Tool Validation

EX-12 paired the project's parser with `go-apfs identitydump` against the same `/dev/rdiskN`. The probe:

1. built a fresh proof fixture,

2. ran the project's parser, recording every resolved (`oid`, `paddr`, `xid`) sample published by the OMAP,
3. replayed object-header validation rules in Python at each published `paddr`,
4. ran `go-apfs identitydump` against the same raw container,
5. compared the two tools' agreement on the FS-tree root `oid`.

The two tools agreed on the FS-tree root OID. They disagreed on which scan XID to select: this project's parser picked the highest visible XID; `go-apfs` picked one lower. The divergence is recorded as a caveat (`go_apfs_active_state_observation`); both tools' OMAP lookups internally use a consistent scan XID and produce internally consistent root resolutions, so the cross-tool oracle survives.

Two tools that disagree on selection but agree on the OID reached through that selection remain useful as oracles for each other, provided the selection difference is recorded.

4.6 What The OMAP Does Not Do

The OMAP does not resolve objects *within* the FS tree. FS-tree internal-node entries also carry OIDs, and those OIDs require OMAP resolution at the scan XID — through the *volume* OMAP, not the container OMAP. The next chapter develops this rule and the fixture that surfaced it.

Part III

The File-System Tree

Chapter 5

The File-System Tree

5.1 The Rule

Observation. An FS-tree internal-node entry carries an 8-byte value. That value is a *virtual object identifier*, not a physical block address. To read the child node, the walker must resolve the value through the *volume* OMAP at the scan XID. A walker that reads the value as a *paddr* reads random blocks on any FS tree of depth ≥ 1 .

The Apple reference documents that FS-tree objects live in the volume’s virtual address space, but does not foreground that *internal-node child references are members of the same address space*. The reference reads naturally as “the internal value is a pointer into the same area as physical extents,” and many B-tree formats indeed encode internal references as direct physical offsets. The converged open-source code is unanimous in the right direction, but the unanimity is implicit: each reader’s B-tree walker calls an object resolver that internally performs the OMAP lookup. A reimplemented walker without an equivalent resolver does not surface the bug on a small fixture, because a small enough FS tree fits in a single leaf node and has no internal entries.

EX-10 through EX-12 fixtures were each small enough to avoid the case. EX-14 produced a fixture dense enough to require an internal level, and the parser broke.

5.2 The Probe (EX-14, EX-15)

Observation. EX-14 reported APFS object validation failed: `checksum mismatch at block 1031` and aborted before publishing a selected checkpoint. The probe `fsck_apfs -n` returned clean; `go-apfs identitydump` succeeded on the same container. The image was not malformed.

EX-15 replayed the upstream context in Python, validating every candidate against the strict rules of chapters 3 and 4. Every NXSB candidate validated. Every checkpoint map block validated. The container OMAP, volume superblock, and volume OMAP all validated. The FS-tree root validated. The walker then dereferenced an internal entry whose 8-byte value was 1031, read block 1031, and encountered 4096 bytes of zero — an unallocated block — whose Fletcher-64 mismatched.

The volume OMAP, queried for `oid=1031` at the scan XID, returned `paddr=505`. Block 505 was a valid FS-tree internal node. The walker had been asked to follow an OID, and had instead read the OID’s numeric value as a *paddr*.

`walk_fs_node` now takes the volume OMAP resolver and performs an `(oid, scan_xid)` lookup for every internal entry before reading the child. A missing OMAP mapping returns a typed `InvalidObject` hard stop naming the OID.

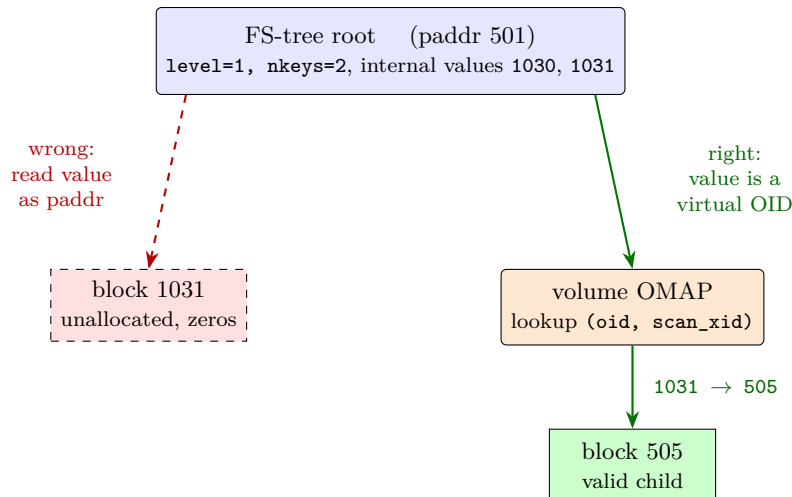


Figure 5.1: The headline finding. An FS-tree internal-node value is a virtual OID; the walker must consult the volume OMAP at the scan XID before reading the child. Reading the value as a paddr lands on an unallocated block.

Two synthetic regression tests close the bug:

1. `fs_tree_internal_value_is_virtual_oid_resolved_via_omap` builds an 8-block synthetic image where the FS-tree root's internal entry has value 1030. With the fix, the walker follows OMAP to the correct paddr and reads the leaf record. Without the fix, the walker short-reads on block 1030.
2. `fs_tree_internal_oid_missing_from_omap_is_hard_stop` asserts that an internal value pointing at an OID with no OMAP mapping returns a typed error mentioning the missing OID rather than silently skipping or attempting a paddr fallback.

5.3 Object-Header Validation

Two details follow from the FS-tree root being virtual.

Observation. The FS-tree root is reached through the volume OMAP, so on disk it carries a virtual `obj_phys_t` header. The storage flags are `0x0000_0000` (virtual), not `OBJ_PHYSICAL`.

Observation. A virtual object's `o_oid` is independent of the block's physical address. The validator must not require `o_oid == paddr` for virtual objects; the correct check is `o_oid == omap_lookup.oid`.

The validator therefore has two modes: `ExpectedStorage::Physical`, which requires `o_oid == paddr`, and `ExpectedStorage::Virtual`, which does not. Every call site picks one; a block validated under the wrong mode is rejected.

5.4 Required Record Families

An indexer that reproduces POSIX namespace plus logical, allocated, and clone-deduplicated size needs the following record families.

DIR_REC. Directory entries. Key carries the parent inode’s file ID and the name hash; value carries the child file ID and entry type. The directory structure is built by walking `DIR_REC` records.

INODE. File, directory, symlink, and other entry metadata. Carries the parent ID, link count, mode, internal flags, and an xfield tail.

XATTR. Extended attributes. The indexer cares about three specifically: `com.apple.fs.symlink` (the symlink target), `com.apple.decmpfs` (the compression metadata header), and `com.apple.ResourceFork` (out of the defended scope but tracked for the decmpfs fail-closed gap).

SIBLING_LINK, SIBLING_MAP. Hard-link unification. A directory entry that participates in a hard-link group carries a sibling ID xfield; `SIBLING_MAP` closes the mapping from sibling ID to file ID.

dstream fields. Logical and allocated size sources. Carried either inside `INODE`’s xfield tail as `INO_EXT_TYPE_DSTREAM` or as a dedicated `j_dstream_id_val_t` record. The `j_dstream_id_val_t` also carries a reference count used by chapter 8’s clone-deduplication.

file_extent. Per-inode extent records, keyed by `dstream_id`. Required for clone-dedup (chapter 8); not consumed by namespace or logical size.

phys_ext. Extent-reference-tree records. One per shared physical extent, carrying (`paddr_first`, `length_blocks`, `refcnt`, `owning_obj_id`). The extent-reference tree is a separate B-tree, distinct from the FS tree, and is opened only when clone-dedup is wanted. On detached `.dmg` images the tree’s storage class is physical; on live volumes it may be virtual.

`snapshot_metadata` and `snapshot_name` records are decoded but not consumed; an “unsupported” bucket catches record types the parser does not recognise. A record in the unsupported bucket on an allowlisted volume triggers a typed validation gap.

5.5 Topology Summary

To enumerate one volume, the walker performs:

1. Container OMAP lookup at scan XID for the volume’s superblock OID, returning the volume superblock `paddr`.
2. Volume superblock decode: the feature allowlist is enforced; unknown incompatible bits, encryption, sealed-volume bits, and normalization-insensitive bits force `Unsupported(reason)`.
3. Volume OMAP open; same encryption / unknown-flag gates as chapter 4.
4. Volume OMAP lookup at scan XID for the FS-tree root virtual OID, returning the root `paddr`.
5. FS-tree root validation as a virtual B-tree node.
6. Recursive descent: at each internal node, an OMAP lookup per child entry; at each leaf node, decode of every record per the families above.

Every FS-tree reader in the Rust crate goes through this path; no code path treats an internal value as a `paddr`. The next chapter covers the variable-length tail of the leaf records.

Chapter 6

Record Bodies and the Xfield Cursor

An FS-tree leaf record consists of a fixed-size header and a variable-length tail of extended fields. The header decoding is mechanical. The tail follows one cursor rule, applied uniformly across every record family that uses xfields.

6.1 The Cursor Rule

Observation. The xfield tail follows one cursor rule, derived from `linux-apfs-rw`, `apfs-fuse`, `dissect.apfs`, `go-apfs`, `libfsapfs`, and the Sleuth Kit:

1. The tail begins with `xf_blob_t`: 4 bytes containing `xf_num_exts` and `xf_used_data`.
2. Immediately after is the metadata table: `xf_num_exts` entries of `x_field_t`, each 4 bytes (`x_type`, `x_flags`, `x_size`).
3. The value area begins immediately after the metadata table. There is no extra alignment of the value-area start.
4. Reading in metadata-table order, each value consumes `round_up(x_size, 8)` bytes. The cursor advances by the padded length, not by `x_size`.
5. `xf_used_data` equals `sum(round_up(x_size, 8))` for the record's xfields.

One rule across all xfield-bearing record families. A record whose `xf_used_data` disagrees with the padded sum is malformed.

The rule fits in a few lines of code:

```
1 let mut cursor = metadata_end;
2 let mut padded_total = 0usize;
3 for entry in metadata.iter() {
4     let value = &blob[cursor .. cursor + entry.x_size as usize];
5     let padded = (entry.x_size as usize + 7) & !7;
6     cursor += padded;
7     padded_total += padded;
8 }
9 assert_eq!(padded_total, blob_header.xf_used_data as usize);
```

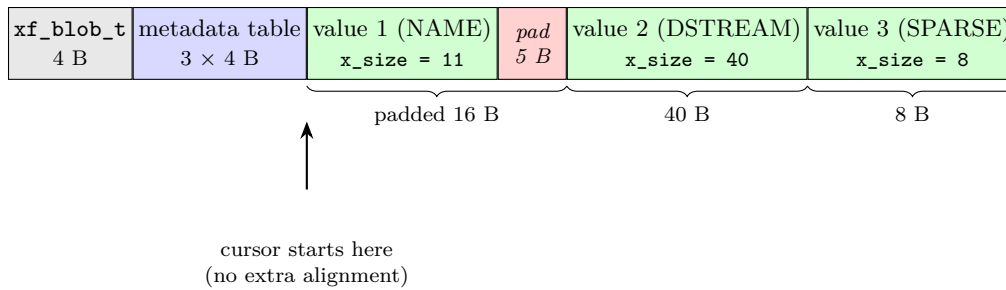


Figure 6.1: The xfield tail for the sparse inode at OID 23 (`xf_num_exts=3`, sizes 11/40/8). The cell widths are proportional to bytes. The value cursor starts immediately after the metadata table; each value advances by `round_up(x_size, 8)`. Total padded value-area length is $16+40+8 = 64$ B, matching `xf_used_data`.

6.2 Where The Reference Reads Otherwise

Apple’s reference admits a reading under which the value area is independently eight-byte aligned: the cursor would jump to the next eight-byte boundary after the metadata table before reading the first value. The difference is invisible when `xf_num_exts * 4` is already a multiple of eight, but diverges otherwise.

The converged code performs no such re-alignment. The value cursor starts at the metadata-table end; each value’s padding to the next eight-byte multiple is performed *after* the value, not before the first.

EX-13 initially modelled four candidate layouts and selected per-record. The probe recovered the correct values but did not explain why. SR-015 reviewed the converged sources and showed the four candidates were a probe artefact: one rule explains every record. EX-16 re-ran the decode under the single rule and confirmed it.

6.3 Evidence From The Proof Fixture

Observation. On the proof fixture, 14 records carry xfield tails. Under the single cursor rule, every one of them satisfies `xf_used_data == sum(round_up(x_size, 8))`, and every one of them reconstructs the namespace and per-file logical size identically to the mounted POSIX oracle.

The records and their xfield shapes:

oid	family	num_exts	x_size set	xf_used_data	padded sum
2	inode	1	5	8	8
3	inode	1	12	16	16
16	inode	1	4	8	8
17	inode	1	4	8	8
17	dir_rec	1	8	8	8
17	dir_rec	1	8	8	8
18	inode	1	11	16	16
19	inode	2	15, 40	56	56
20	inode	2	10, 40	56	56
23	inode	3	11, 40, 8	64	64
24	inode	2	40, 10	56	56
26	inode	1	9	16	16
27	inode	2	17, 40	64	64
28	inode	2	17, 40	64	64

The sparse inode at OID 23 is the load-bearing row. Its three xfields (**NAME=11**, **DSTREAM=40**, **SPARSE_BYTES=8**) have padded lengths summing to 64. The **NAME** value’s non-multiple-of-eight **x_size** forces the cursor to use padded advancement. Earlier candidate-scoring needed per-record reasoning here; the single rule decodes it cleanly.

No record has unused trailing bytes inside its xfield blob.

6.4 The Required Xfield Types

The xfield universe is large; an indexer needs four types and tolerates others.

INO_EXT_TYPE_NAME. The inode name. The name is also reachable through **DIR_REC** keys, and the indexer prefers **DIR_REC** names for namespace construction (they carry the directory placement). The inode-name xfield is a consistency check.

INO_EXT_TYPE_DSTREAM. A `j_dstream_t` embedded in the inode, carrying logical size, allocated size, default crypto id, and a default-extent reference.

INO_EXT_TYPE_SPARSE_BYTES. A count of bytes the sparse-allocation layer has not committed to disk. This is not a logical size; chapter 8 develops the distinction.

DREC_EXT_TYPE_SIBLING_ID. A directory record’s sibling-group identifier for hard links. **SIBLING_MAP** closes the mapping from sibling ID to inode.

Other types (**INO_EXT_TYPE_FS_UUID**, **INO_EXT_TYPE_PURGEABLE_FLAGS**, and so on) appear in the wild but are not consumed by the indexer. The cursor rule reads past them correctly — every value is opaque bytes of length **x_size** padded to a multiple of eight — and only the values the parser inspects need a known type.

6.5 Field-Level Cross-Tool Equality

The xfield rule, the fixed-header decoders, and the fail-closed gates of the next chapter together specify the body decoder completely. The project verified the specification by running two independent decoders against the same raw container and comparing every field.

Observation. On the proof fixture, the Rust body decoder emits 53 records. An independent Python decoder, built from the same SR-015 rule and the same fail-closed boundary, emits 53 records. The two record lists key identically on (`node_paddr`, `entry_index`). Per-record field comparison finds zero divergent fields across `key`, `value`, every `xfields[*]`, `xfield_used_data`, `xfield_padded_total`, `xfield_unused_trailing_bytes`, `validation_notes`, `key_len`, `value_len`, `object_id`, `raw_type`, and `family`.

The Python decoder and the Rust decoder are independent implementations of the same SR-015 / SR-016 rules; their agreement makes the rules themselves testable. A future `go-apfs-body-dump` probe would add a third oracle.

A reader implementing an APFS body decoder applies, in order: the fixed header for the record type (Apple reference), the cursor rule above to the xfield tail, every fail-closed condition in chapter 7, and a cross-tool comparison against an independent implementation.

Chapter 7

The Fail-Closed Boundary

The parser refuses to emit a row it does not understand. The cost of a silently wrong row is unbounded (it propagates into aggregates, treemaps, and user actions); the cost of hard-stopping with a typed reason is bounded (the caller dispatches to the fallback). The discipline is concentrated in two chokepoints.

7.1 The Two Chokepoints

Object headers. One function (`validate_object_block` in the Rust crate) validates every block before any reader trusts its body. It checks the Fletcher-64 checksum, the object type, the expected storage class, the encryption flag, the no-header flag, optional `o_oid == paddr` for physical objects, and the optional max-XID guard.

Record bodies. One function (`decode_fs_record`) validates every FS-tree leaf record against the rules below before any emission path trusts the decoded values.

No second path bypasses either chokepoint: a block that fails header validation never reaches a body decoder; a record that fails body validation never reaches the namespace builder. A representative hard stop, from the body decoder:

```
1 if blob_header.xf_used_data as usize != padded_total {
2     return Err(ScanError::InvalidObject(format!(
3         "xf_used_data {} disagrees with padded value-area sum {}",
4         blob_header.xf_used_data, padded_total,
5     )));
6 }
```

The error carries the rule that failed, the observed value, and the expected value. The caller dispatches on the variant (`InvalidObject`); the substring is preserved for diagnostic display.

7.2 The Per-Record Catalogue

Observation. Twenty-one per-record fail-closed cases are enforced by Rust unit tests on synthetic raw blocks. Each test asserts that the body decoder returns a typed `InvalidObject(reason)` with the rule's substring.

7.2.1 Short bodies

1. Inode value shorter than the fixed `j_inode_val_t` struct.
2. Directory-record value shorter than the fixed `j_drec_val_t` struct.
3. Sibling-link value with `name_len` exceeding the bytes actually available.
4. Sibling-map value shorter than 8 bytes.

7.2.2 Malformed names

5. Variable-length name length longer than the bytes available.
6. Embedded NUL inside a required name (other than the null-terminator).
7. Non-UTF-8 bytes in a required name.
8. Directory-entry type outside the POSIX `DT_*` allowlist.

7.2.3 Malformed xfields

9. Duplicate xfield types inside one blob.
10. `xf_used_data` disagreeing with `sum(round_up(x_size, 8))`.
11. Xfield value cursor running past the blob.
12. Xfield metadata table running past the blob.
13. Xfield blob shorter than the 4-byte `xf_blob_t` header.
14. Required xfield with the wrong `x_size` (DSTREAM of length other than 40).
15. Required xfield with the wrong `x_size` (SIBLING_ID of length other than 8).

7.2.4 Malformed xattrs

16. Xattr value shorter than the fixed header.
17. Xattr sets both `EMBEDDED` and `STREAM` flags.
18. Xattr sets neither flag.
19. Xattr sets unknown flag bits.
20. Xattr body length disagrees with `xdata_len`.
21. Xattr stream body shorter than `j_xattr_dstream_t`.

7.3 The Cross-Record Cases

Three SR-016 cases depend on consistency across more than one record and therefore cannot be enforced by synthetic single-record tests. They are deferred to fixture-level probes; the open questions chapter tracks their status.

Drec type vs inode mode. A directory entry’s `DT_*` type and the referenced inode’s `S_IFMT` mode must agree. A regular-file inode pointed at by a `DT_DIR` entry is malformed at the volume level even though both records validate in isolation.

Sibling-map closure. A directory entry that carries `DREC_EXT_TYPE_SIBLING_ID` requires a `SIBLING_MAP` record that maps that sibling ID to the entry’s child file ID. A missing closure produces a hard-link group with a dangling edge.

Compressed-size precedence conflict. An inode that sets `INODE_HAS_UNCOMPRESSED_SIZE` but does not carry a `com.apple.decmpfs` xattr is internally inconsistent at the cross-record level. Chapter 8 develops the precedence rule that prevents the conflict from producing wrong rows.

These cases will close as the project lands EX-22 and the future cross-record consistency probe.

7.4 Why The Catalogue Is Worth Its Weight

The catalogue exercises in three not-ordinary cases:

Format drift. APFS reserves bits and types over time. A container written by a newer macOS may carry an xattr flag, an xfield type, or an incompatible-feature bit the parser does not recognise. The unknown-bit hard stops mean a newer container produces a typed refusal rather than a quietly wrong row.

Genuinely malformed images. Recovery scenarios, partial image dumps, or test corpora may include containers with structurally inconsistent records. The catalogue refuses them with a precise reason, which the caller can present as a diagnostic.

Parser bugs. If a future change to the body decoder silently re-introduces an off-by-one in the xfield cursor, the `xf_used_data` structural assertion catches it on the first record with a non-multiple-of-eight `x_size`. The catalogue is also a regression suite.

7.5 Object-Header Storage Classes

`validate_object_block` takes the expected storage as a parameter; a block validated under the wrong class is rejected.

Physical. The object’s `o_oid` equals its physical block address. Container superblocks, checkpoint maps, OMAP nodes, and free-queue nodes are physical.

Virtual. The object lives in a virtual address space resolved by an OMAP. `o_oid` is independent of the block’s `paddr`; the validator checks `o_oid == omap_lookup.oid` instead. Volume superblocks (looked up through the container OMAP), FS-tree roots and internal nodes (looked up through the volume OMAP), and dstream extent nodes are virtual.

Ephemeral. In-memory between transactions; reached through the checkpoint map. The parser uses the checkpoint map’s mapping, never an ephemeral storage flag.

Mixing classes is a structural mistake. Chapter 3 records that checkpoint-map blocks are themselves physical despite naming ephemeral objects; chapter 5 records that FS-tree roots are virtual despite living at a known paddr. Both rules are isolated to their call sites.

When the boundary refuses a source, the caller dispatches to the POSIX-traversal fallback (chapter 11), which produces the same row shape through supported APIs.

Part IV

Names and Sizes

Chapter 8

Sizes and Their Precedences

APFS stores file size in several places, and the places do not always agree. The parser emits three per-inode metrics, each with its own precedence rule and its own oracle:

Logical. The user-visible byte length (`st_size`). The primary anchor; covers every case.

Allocated. Bytes APFS has assigned to the inode’s content (`st_blocks * 512`). Covers every shape the macOS write path produces.

Real. Clone-deduplicated allocated bytes. Each inode is charged its share of physically-allocated bytes, accounting for extents that multiple inodes reference. Validated by the extent-reference tree.

The chapter develops them in that order.

8.1 Logical Size

Logical size is the user-visible byte length: the number POSIX `st_size` reports and the number `ls -l` prints. It has a stable oracle, applies uniformly to ordinary, sparse, cloned, hard-linked, symlink, and compressed files, and is the size every user-facing tool agrees on.

8.1.1 The Precedence Rule

Observation. Five steps, applied per inode, produce `st_size` for every case the validated corpus covers:

1. If the inode’s `internal_flags` carry `INODE_HAS_UNCOMPRESSED_SIZE`: pick `inode.uncompressed_size`.
2. Else if the inode carries a `j_dstream_t` (via `INO_EXT_TYPE_DSTREAM` or a paired `j_dstream_id_val_t` record): pick `j_dstream_t.size`.
3. Else if a `com.apple.decmpfs` xattr is present and step 1 did not fire: pick the decmpfs header’s `uncompressed_size` field.
4. For symlinks (`S_IFLNK`): pick the byte length of the `com.apple.fs.symlink` xattr payload.
5. Otherwise: zero.

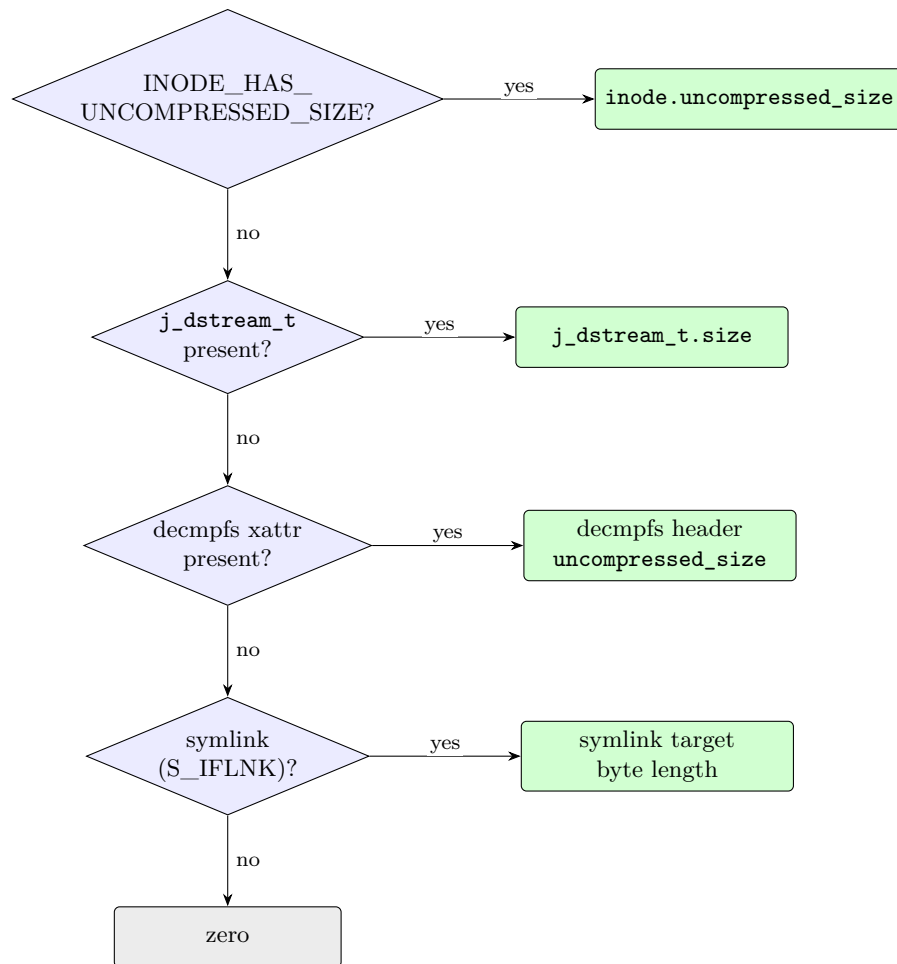


Figure 8.1: Per-inode logical-size precedence. The order is non-negotiable; the compressed case is the load-bearing reason step 1 must beat step 3.

The order matters. Steps 1 and 3 both reach into compression metadata, but they reach for different fields and have different trust; the load-bearing case below shows why.

In pseudocode the same rule reads as one short function:

```

1 def logical_size(inode, xattrs):
2     if inode.internal_flags & INODE_HAS_UNCOMPRESSED_SIZE:
3         return inode.uncompressed_size
4     if inode.dstream is not None:
5         return inode.dstream.size
6     decmpfs = xattrs.get("com.apple.decmpfs")
7     if decmpfs is not None:
8         return decmpfs.header.uncompressed_size
9     if S_ISLNK(inode.mode):
10        return len(xattrs["com.apple.fs.symlink"].payload)
11    return 0
  
```

8.1.2 The Load-Bearing Case: Compression

Observation. On the EX-19 fixture the compressed file (`compressed.txt`, created via `ditto -hfsCompression`) presents:

- `INODE_HAS_UNCOMPRESSED_SIZE` set, `inode.uncompressed_size` = 53248.
- No `j_dstream_t`. APFS stores small compressed data inline in the `decmpfs xattr` and does not allocate a separate `dstream` for it.
- `com.apple.decmpfs xattr` present, with header `uncompressed_size` = 437 and `compression_type` = 0.
- Mounted POSIX `st_size` = 53248.

The `decmpfs` header’s `uncompressed_size` is a placeholder for this layout. Step 1 must beat step 3 or the parser emits 437 instead of 53248. The principle is general: when APFS sets the `inode` flag, the `inode` flag is the source of truth; the `decmpfs` header is metadata the running system maintains for its own decompression path and is not reliably the file’s size.

8.1.3 Sparse Bytes Is A Hint, Not A Size

Observation. `INO_EXT_TYPE_SPARSE_BYTES` is an *allocation hint*, not a logical size. It records the bytes the sparse-allocation layer has not committed to disk.

On the proof fixture’s sparse file:

- `j_dstream_t.size` = 1052897 (which equals `st_size`).
- `INO_EXT_TYPE_SPARSE_BYTES` = 1032192.
- `st_size` = 1052897.

`SPARSE_BYTES` records the difference between the logical extent and the allocated bytes. It is the load-bearing input to the allocated-size rule (next section); it is not the answer to “how large is this file.” Treating `SPARSE_BYTES` as anything else inflates the logical size of every sparse file.

8.1.4 Per-Inode Results

Observation. On the EX-19 same-run fixture covering ordinary, sparse, clone, hard-linked, symlink, and compressed cases, the precedence rule reproduces `st_size` for every unique inode:

path	inode	step picked	value	st_size
ordinary.txt	16	<code>j_dstream_size</code>	29	29
sparse.bin	19	<code>j_dstream_size</code>	1052897	1052897
clone.txt	20	<code>j_dstream_size</code>	29	29
link.txt	23	<code>symlink_target_len</code>	12	12
compressed.txt	24	<code>inode_uncompressed_size</code>	53248	53248

Zero mismatches. The compressed case picks step 1; everything else picks step 2 except the symlink (step 4). The hard link does not appear as a separate row because the rule is per inode and `hard.txt` shares inode 16 with `ordinary.txt`.

8.2 Allocated Size

Per inode, the bytes APFS has assigned to the inode’s content stream. Oracle: `st_blocks * 512`.

Observation. The rule covers every shape the macOS write path produces. Ordinary, cloned, hard-linked, and symlink cases emit `j_dstream_t.allocated_size` (zero for symlinks). The sparse and decmpfs cases were promoted from fail-closed to oracle-validated by EX-26: sparse emits `allocated_size - sparse_bytes`; decmpfs emits the sum of any stream-backed xattrs’ `allocated_size` (the primary dstream is usually absent; embedded xattrs contribute zero because their bytes live inline in the xattr payload).

8.2.1 The Precedence Rule

Observation. Six branches, applied per inode:

1. Regular file with a `j_dstream_t` xfield and no `INO_EXT_TYPE_SPARSE_BYTES` xfield and no `com.apple.decmpfs` xattr: emit `Some(j_dstream_t.allocated_size)`.
2. Regular file with a `j_dstream_t` xfield and a `SPARSE_BYTES` xfield (no decmpfs): emit `Some(allocated_size - sparse_bytes)` (EX-26 Hypothesis A, see §8.2.3).
3. Regular file with a `com.apple.decmpfs` xattr: emit `Some(primary + decmpfs.stream_dstream.allocated + ResourceFork.stream_dstream.allocated)` (each term defaults to zero when absent or embedded; EX-26 Hypothesis F, see §8.2.4).
4. Symlinks: emit `Some(0)`.
5. Directories: emit `Some(0)`.
6. Anything else: emit `None`.

`None` is not a synonym for zero. A row with `allocated_size = None` is recorded in the run’s `not_claimed` register; the metric toggle in the visualisation greys out for that row; any directory aggregate containing a `None`-allocated inode is itself emitted as `None`. A partial aggregate is forbidden, since a consumer might otherwise treat it as authoritative.

8.2.2 Why `allocated_size` Is The Only Candidate

A survey of six independent open-source readers (`linux-apfs-rw`, `apfsprogs/apfsck`, the Sleuth Kit, `libfsapfs`, `dissect.apfs`, `apfs-fuse`, `go-apfs`) finds all six decode `j_dstream_t.allocated_size`; only TSK and `linux-apfs-rw` surface it to callers; no reader surfaces the alternative candidate (the sum of `j_file_extent_val_t.len`) as a public allocated-bytes number. The extent-reference tree is the per-volume reference-counting authority for clones and snapshots, not a per-file allocated-bytes table; it is used here for the Real metric below.

The two halves of the same upstream project disagree: `linux-apfs-rw` (kernel) writes `allocated_size = round_up(ds_size, blocksize)`; `apfsck` (filesystem checker) enforces `allocated_size == sum(j_file_extent_val_t.len)`. The two agree only when every extent ends at `ds_size` and the file has no truncate residue. The right oracle is the external public number (`st_blocks * 512`), not internal field consistency.

8.2.3 The Sparse Identity (EX-22, EX-26)

EX-22 left sparse fail-closed because the fixture carried one sparse shape and the observed identity $\text{allocated_size} - \text{sparse_bytes} = \text{st_blocks} * 512$ needed corpus-level confirmation. EX-26 ran the probe on four sparse shapes; the identity held exactly across all of them.

Observation. On the EX-22 fixture the sparse file carries:

- `j_dstream_t.size` = 1052897 (equals `st_size`).
- `j_dstream_t.allocated_size` = 1056768 (= `round_up(size, 4096)`).
- `INO_EXT_TYPE_SPARSE_BYTES` = 1032192 (the unallocated bytes inside the logical extent).
- `st_blocks` = 48, so `st_blocks * 512` = 24576.

`allocated_size` overstates the actual on-disk allocation by exactly $1,056,768 - 24,576 = 1,032,192$ bytes — the sparse-bytes hint. APFS on macOS follows the `linux-apfs-rw` kernel rule rather than the `apfsck` sum-of-extents rule. The emission rule must therefore split the dstream case by presence of the `SPARSE_BYTES` xfield.

EX-26 corpus result. The identity $\text{allocated_size} - \text{sparse_bytes} = \text{st_blocks} * 512$ holds exactly across four sparse shapes:

shape	allocated_size	sparse_bytes	difference	st_blocks * 512
1 MiB HEAD/TAIL hole	1,056,768	1,032,192	24,576	24,576
10 MiB hole	10,485,760	10,452,992	32,768	32,768
50 MiB hole	52,428,800	52,396,032	32,768	32,768
4 KiB-data-per-64 KiB chunked	2,097,152	1,572,864	524,288	524,288

The branch now emits `Some(allocated_size.saturating_sub(sparse_bytes))`, with saturation guarding the (not observed in practice but algebraically possible) case where the kernel’s residue exceeds the dstream- reported allocation.

8.2.4 The decmpfs Rule (EX-26)

EX-22 left decmpfs fail-closed because no open-source reader agreed on the macOS oracle and the fixture carried only one decmpfs shape. EX-26 added a 256 KiB compressible payload that ditto routes into the resource-fork-stored shape. Both shapes satisfy one rule: allocated bytes equal the sum of any stream-backed xattrs’ `allocated_size` plus the (typically absent) primary dstream `allocated_size`.

Observation. On the EX-26 fixture the two decmpfs files carry:

- `compressed.txt` (52 KiB compressible JSON): no primary dstream; `com.apple.decmpfs` is *stream-backed* (flags = 0x01) with `stream_dstream.allocated_size` = 4096; `com.apple.ResourceFork` is stream-backed with `stream_dstream.allocated_size` = 0 (empty fork). Sum: 4096. Oracle `st_blocks * 512` = 4096.
- `compressed-big.bin` (256 KiB compressible binary): no primary dstream; `com.apple.decmpfs` is *embedded* (flags = 0x02) with payload `fpmc` magic, `compression_type` = 8 (lzvn resource-fork-stored), `uncompressed_size` = 262144; `com.apple.ResourceFork` is stream-backed with `stream_dstream.allocated_size` = 4096. Sum: 4096. Oracle `st_blocks * 512` = 4096.

The decmpfs header’s `compression_type` byte tells the reader *where* the compressed bytes live (xattr inline vs. resource fork). The bytes themselves are always counted by summing the `stream_dstream.allocated_size` fields of every xattr that is stream-backed. Embedded xattrs contribute zero because their bytes live inline in the xattr payload, which is part of the inode body’s storage and does not allocate separate extents.

The disagreement EX-22 documented between `linux-apfs-rw` (publishes uncompressed size / 512) and `apfs-fuse` (publishes zero) resolves against the macOS oracle: neither is right; the sum-of-stream-dstreams formula reproduces `st_blocks * 512` exactly.

The incompressible-payload shape, where ditto leaves `UF_COMPRESSED` off and emits no `com.apple.decmpfs` xattr, falls through to branch 1 and emits `Some(allocated_size)` under the EX-22 baseline. The EX-26 `compressed-random.bin` case does exactly this.

8.2.5 Per-Inode Results

Observation. On the EX-26 fixture, all ten rows match the `st_blocks * 512` oracle exactly:

path	branch	rule	picked
ordinary.txt	1	<code>allocated_size</code>	4096
clone.txt	1	<code>allocated_size</code>	4096
compressed-random.bin	1	<code>allocated_size</code>	131,072
sparse.bin	2	<code>allocated_size - sparse_bytes</code>	24,576
sparse-medium.bin	2	<code>allocated_size - sparse_bytes</code>	32,768
sparse-large.bin	2	<code>allocated_size - sparse_bytes</code>	32,768
sparse-chunked.bin	2	<code>allocated_size - sparse_bytes</code>	524,288
compressed.txt	3	<code>decmpfs.stream_dstream.allocated</code> (xattr-stream-stored)	4096
compressed-big.bin	3	<code>ResourceFork.stream_dstream.allocated</code> (resource-fork-stored)	4096
link.txt	4	<code>zero</code> (symlink)	0

Zero mismatches; every row picks a value the rule emits, against the macOS oracle. The hard-linked file `hard.txt` does not appear as a separate row because it shares its inode with `ordinary.txt` and contributes through the same per-inode rule.

8.3 Real (Clone-Deduplicated) Size

Allocated size charges each clone its full dstream. The volume’s on-disk allocation is smaller: clones share extents, and a survey of a clone-heavy directory using allocated bytes overstates the disk usage. The *real* size charges each inode its share of physically-allocated bytes, so a directory total equals the unique on-disk bytes in its subtree.

Oracle. Per-inode `du -A` on macOS reports apparent size (it reads `st_blocks * 512` via `lstat`, which does not dedup clones), so there is no public per-file macOS oracle. The on-disk authority is the extent-reference tree itself: $\sum \text{phys_ext.length} \cdot \text{block_size}$ is the ground truth for the volume’s allocated bytes.

8.3.1 The Formula (EX-27)

Three APFS trees feed the computation: the FS-tree’s `inode` records, the FS-tree’s `file_extent` records (keyed by `dstream_id` — clones share a dstream, not an inode), and the extent-reference

tree’s `phys_ext` records (one per shared physical extent, carrying (`paddr_first`, `length_blocks`, `refcnt`, `owning_obj_id`)).

1. For each `file_extent`, split its physical byte range against the overlapping `phys_ext` records. A `phys_ext` record’s `refcnt` is the count of `file_extents` pointing at that range; sub-extents not covered by any `phys_ext` record have implicit `refcnt = 1`.
2. Per dstream: $\text{dstream_dedup} = \sum_i \text{sub_extent}_i.\text{length} / \text{sub_extent}_i.\text{refcnt}$.
3. Per inode: $\text{real_size} = \text{dstream_dedup} / \text{dstream.refcnt}$, where `dstream.refcnt` is the count of inodes sharing this dstream (from the `j_dstream_id_val_t` record).

Invariants. On the EX-27 fixture (three clone families: 5-way, 3-way, and a partial-share rewrite):

- $\sum_{\text{dstreams}} \text{dstream_dedup} = \sum_{\text{phys_ext}} \text{length_blocks} \cdot \text{block_size}$ exactly.
- $\sum_{\text{inodes}} \text{real_size}$ equals the same total modulo integer-division rounding of $\leq (\text{refcnt} - 1)$ bytes per clone-shared dstream (2-byte residue on the EX-27 fixture across two shared dstreams).

The second invariant is what makes the metric sum cleanly across the treemap: per-clone shares summed over a directory equal the unique on-disk bytes in that subtree.

8.3.2 Storage Class Of The Extent-Reference Tree

Observation. On detached `.dmg` images `hdiutil` creates, the extent-reference tree carries `OBJECT_TYPE_PHYSICAL` storage — the OID is the root `paddr` directly. Live volumes can use virtual storage, in which case the volume OMAP returns the root `paddr`. The crate reads `extentref_tree_type_raw` on the volume superblock and dispatches; a virtual tree with no OMAP mapping at the scan XID records a validation gap.

8.3.3 Fallback Behaviour

The POSIX-traversal fallback has no access to per-extent refcounts. It emits `real_size = allocated_size`: the fallback walker cannot dedup, and saying so is more honest than publishing the wrong number under the same column heading.

8.4 Directory Aggregates

A directory’s total is the *unique-inode logical sum within the aggregate root*.

A regular file’s logical size contributes to a directory total once, regardless of how many hard-linked paths within that directory tree reference the same inode. A symlink contributes its target byte length to its own row but contributes nothing to any directory total (the target may live elsewhere or not exist at all, and even when it exists its size is already counted at the target’s row).

Why this policy. The naive policy — sum the logical size of every path under a directory — catastrophically over-counts on real directories with hard-linked content: mail spools, package caches, Git object stores, snapshot archives. The user looking at a 12 GB directory deserves to know that 11 GB of it is a single 11 GB file that also appears elsewhere, not that the directory

legitimately holds 12 GB of unique content. WizTree-on-NTFS rarely encounters hard-link density that exposes this; on APFS the case is common.

Consequence. Sibling directory totals are not strictly additive into their parent when the same file identity appears in more than one subtree. This is correct, not buggy, but the user interface must label aggregate semantics so the user understands what the number means.

The aggregate rule is implemented in one place (`aggregate.py`, ported to the Rust crate’s `namespace` module) using `setdefault(file_id, logical_size)` keyed by inode — the policy is enforced by construction rather than by careful summing.

Allocated-size aggregates use the same unique-inode rule with a `None`-collapse step: any contributing inode whose `allocated_size` is `None` (any case not covered by the EX-26 rules) propagates `None` up to the directory total. Real-size aggregates follow the same shape: per-inode shares sum across the directory, and a `None` contribution (from the fallback walker, or from a raw run where the extent-reference tree did not validate) collapses the total.

8.5 Out Of Scope

Two metrics remain outside the chapter’s defended scope; chapter 13 records each as an open question.

Exclusive size. Bytes owned only by this inode after accounting for clones *and* snapshots. The chapter’s Real metric handles clones (extent-reference tree); the snapshot-metadata tree must be joined for full exclusivity.

Shared size. Bytes shared with at least one other inode or snapshot. The complement of exclusive.

Chapter 9

Names Across Volume Modes

Path names come from directory record keys, stored as UTF-8 byte strings. The chapter develops one positive finding (verbatim preservation) and one deliberate scope limit (lookup-by-name is not claimed).

9.1 Verbatim Preservation

Observation. The Rust body decoder emits the bytes the directory key stores. The parser does not normalise, case-fold, or interpret. POSIX traversal of the same volume returns the same bytes. The two agree byte for byte on both case-insensitive and case-sensitive APFS volumes.

Anything more elaborate — “emit the canonical form,” “convert to NFC” — would be wrong on some volume and would silently rename files in the output.

9.2 Two Volume Modes Exist

APFS volumes come in two case-and-normalisation modes that the indexer must handle.

APFS (case-insensitive). `case_insensitive = true, normalization_insensitive = false.`

The volume’s incompatible-feature mask sets the case-insensitive bit only. The volume is also normalisation-insensitive for lookup, but APFS reserves the explicit incompat bit for the case-sensitive variant. The asymmetry is a quirk of the on-disk format, not a statement about behaviour.

Case-sensitive APFS. `case_insensitive = false, normalization_insensitive = true.`

Both modes appear on real systems, and both have to be supported.

9.3 What The Modes Affect

The modes affect *lookup*: when the kernel resolves a path it consults the volume’s case and normalisation behaviour to decide whether two names refer to the same file. The modes do not affect row *enumeration*, which walks directory keys and returns their bytes. The parser emits correct rows on both modes without performing the lookup-equivalence test, because lookup is the running system’s responsibility.

9.3.1 The Lookup Oracle

Observation. Across the two volume modes, APFS lookup behaves as follows. On the EX-20 same-run fixture using `os.O_CREAT | os.O_EXCL`:

operation	CI volume	CS volume
create <code>plain.txt</code>	accept	accept
create <code>CaseName.txt</code>	accept	accept
<code>casename.txt</code> after <code>CaseName.txt</code>	EEXIST	accept
create NFC <code>café.txt</code>	accept	accept
NFD <code>café.txt</code> after NFC sibling	EEXIST	EEXIST

The case-insensitive volume rejects the lowercase form because lookup considers the two names equivalent. Both volumes reject the decomposed Unicode form after the precomposed sibling because both treat NFC and NFD as the same name for lookup. This is what APFS does; no APFS-specific knowledge in the indexer is needed to produce the result, because the kernel did the work.

9.3.2 Rust vs POSIX Equality

Observation. The Rust-emitted path bytes match POSIX-traversal path bytes exactly on both volumes: `path.encode("utf-8").hex()` produces the same string on both sides for every entry. CI corpus has four entries; CS has five (the case-distinct sibling also appears); equality holds for every row. Both sides walk the directory key storage; POSIX walks through the kernel, Rust walks the FS tree directly; neither applies normalisation, so the bytes are the same bytes.

9.4 Lookup-by-Name Is Not Claimed

The APFS lookup primitive is the directory hash: CRC32C over the name’s UTF-32LE codepoints, with NFD normalisation applied first, and Unicode case-fold applied before normalisation on case-insensitive volumes. Two names with the same hash collide within a directory; APFS resolves the collision by storing both keys and walking them.

A *find this exact path* feature must reproduce that hash. The project has not validated a hash implementation against a same-run oracle, and until it does the parser refuses to find files by name. Row enumeration powers the treemap, sort-by-size, drill-down, and context-menu operations on rows the user already sees; a silently wrong lookup-by-name returns the wrong file for the user’s query. The cost asymmetry dictates the scope asymmetry.

The substring search the native shell ships (chapter 13) operates on already-enumerated names: it does not perform an APFS-hash lookup and therefore inherits no risk from the missing hash.

9.5 Names and Aggregates

Directory aggregates (chapter 8) key on inode identity, not on path. Two paths that hard-link to the same inode contribute that inode’s size once to a directory total, regardless of whether their names are casewise or normalisationally distinct. Hard-link unification reads `SIBLING_LINK` and `SIBLING_MAP` to find every name pointing at a given inode; names are emitted verbatim in each row, the inode is recorded as the row’s `file_id`, and the aggregate keys on the `file_id`.

9.6 What A Future Lookup-by-Name Would Need

1. A same-run hash oracle: take a constructed name, run APFS's hash through a controlled API (the kernel's `O_EXCL` accept/reject is the cheapest indicator), produce positive and negative cases for the hash equivalence class.
2. NFD normalisation tables and Unicode case-fold tables matching the macOS version the volume was written on (tables change across Unicode revisions).
3. Collision handling: a directory may store multiple keys with the same hash; the lookup walks all of them.
4. Volume-mode dispatch: case-insensitive applies case-fold before normalisation; case-sensitive does not.

Part V

Boundaries

Chapter 10

Identity, Reuse, and Cache Keys

A second scan of an unchanged volume should be faster than the first. The natural mechanism is a subtree-summary cache: walk the FS tree once, summarise each node, reuse the summary on the next run when the node has not changed. The chapter develops what identity that cache must key on, why the obvious key is unsafe, and what the shipping CLI cache already does.

10.1 Why OID Alone Is Unsafe

A summary cache keyed on (`volume_uuid`, `oid`) would deduplicate unchanged subtrees across runs. It would also return stale summaries the first time the volume mutates a subtree the cache already knows.

Observation. The FS-tree root’s logical OID stays stable across mutations; every other identity field changes. EX-06 mutated the proof fixture, recorded the FS-tree root identity before and after, and observed:

field	across mutation
logical OID	stable
physical address (<code>paddr</code>)	changed
object XID	changed
object checksum	changed
block hash	changed

A cache keyed on OID would return the pre-mutation summary for the post-mutation tree, and the user would see a treemap that does not match the disk. The OID names *some* version of the object; it does not name particular content. A cache that wants content identity must use a content identifier.

10.2 The Shape Of A Safe Cache Key

A safe cache key distinguishes entries that share an OID but differ in content. The minimum sufficient key is a tuple from several layers:

Source identity. A volume UUID, a container UUID, and an image fingerprint (for detached sources). Distinguishes one APFS volume from another, and distinguishes the same volume’s different image dumps.

Parser version. A monotonic identifier for the body decoder and the namespace builder. A parser change invalidates every summary, because a summary’s interpretation depends on the parser that produced it.

Summary schema version. A monotonic identifier for the summary structure itself. A schema change is independent of a parser change.

OMAP context. Whether the OID was resolved through the container OMAP or the volume OMAP. A collision between the two name spaces is rare but representable.

OID. The virtual identifier.

Object XID. The version of the object the summary was built for.

Physical address. Useful as a cross-check; an OMAP that resolves (oid, xid) to a different paddr than the cache expects invalidates the entry.

Checksum or content hash. The strongest individual signal. A block whose checksum has changed must not reuse a summary keyed on the old checksum.

Expected type and subtype. Defensive. An OID that has been re-allocated to a different object class is a new object.

The key is not a small integer: nine fields, eight of which the parser already computes during a read, the ninth (parser version) constant per build.

10.3 What EX-07 Proved

EX-07 asked a narrow question: under exact node-identity matches on every field of the key above, does subtree reuse ever produce a wrong summary?

Observation. EX-07 found zero false reuse for exact node-identity matches on the detached lab corpus. Two parses that see the same node identity get the same summary; reusing one for the other is safe.

The result is not a production cache theorem. EX-07 ran one corpus, on detached images, with a small mutation set. A production cache that consumes the raw parser must also resolve cache storage with crash-safe writes (a corrupting cache produces stale-and-wrong instead of absent-and-correct) and an explicit invalidation policy covering parser-version mismatch, source identity mismatch, feature drift, checkpoint rollback, snapshot switch, unknown record-body types, and any new metric absent from the pre-existing schema. The Real metric of chapter 8 is the most recent example of a column a pre-existing cache would not have recorded.

10.4 The Shipping CLI Cache (EX-30)

The CLI ships a v1 cache behind the `-cache` flag (off by default). It operates on the fallback walker, not the raw parser: the safe-reuse work above remains future for raw-mode subtree summaries. The v1 design is intentionally smaller than what EX-07 gestures at.

Mechanism. On a cache miss, the full scan runs, the output is written to a content-addressed entry under `/Library/Caches/apfs-fastindex/`, and the entry is keyed on a probe-derived signature of the scanned directory tree. On a hit, the cached entry is replayed without walking the tree.

Observation (EX-30). The cache works on stable trees and does not on actively-churning ones:

target	first scan	rescan (-cache)	speedup
project docs	0.33 s	0.007 s	47×
project root	0.59 s	0.12 s	4.9×
/Users/kai	46.6 s	45.4 s	1.0×

Two factors compound on /Users/kai-class trees. First, the probe uses naive `read_dir` plus `symlink_metadata` per child; the walker uses `getattrlistbulk`, which batches metadata reads. On 172k directories the probe pays roughly 37 s versus the walker’s roughly 9 s — a cache hit costs more than no cache. Second, dozens of subdirectories under /Users/kai mutate continuously (browser caches, Spotlight, log rotation, Xcode derived data); the signature almost never matches the cached signature, every cache write is wasted.

For stable trees (project directories, /Applications, read-only system paths) the v1 cache delivers a 5–50× speedup. The CLI prints `cache: hit` to stderr on hits; the documentation labels the flag “useful for stable trees; not recommended for active home directories.”

10.5 What R4 v2 Needs

The v1 cache becomes useful on home-directory-scale trees when two pieces land. Both belong to future work; neither is in the shipping binary.

1. **Walker-emitted signature.** The parallel walker accumulates each directory’s signature inline, while it is already calling `getattrlistbulk`. The separate directory-only probe disappears; the cache-hit cost drops from roughly 37 s to roughly 0 s.
2. **Per-subtree caching.** One signature per directory rather than per scan. On rescan the walker descends incrementally: when a directory’s signature matches the cached one, the subtree’s entries and aggregates are reused without descent. A few churning subdirectories invalidate themselves; the unchanged 99 % stays cached.

Together these turn a 45-second rescan on /Users/kai into roughly one second when 10 % of subdirectories have changed. That is the R4 v2 promise; an EX-* in the next phase will validate it against a forced full rescan.

10.6 What This Chapter Forecloses

No bare-OID cache. The parser does not key reuse on OID alone, even with parser-version qualification. The mutation behaviour above makes a bare-OID cache unsafe in principle.

No reuse across mismatched scan XIDs. A summary computed at scan XID *a* is not reused at scan XID *b*. Object identity includes the XID; relaxing that turns the cache into a stale-state producer. The cache may compare keys across runs only when the keys agree on every field, scan XID included.

EX-06’s discipline — record every plausibly-load-bearing identity field at every relevant boundary before deciding which ones matter — generalises beyond caching. EX-12’s OMAP probe used

the same habit, recording every resolved (`oid`, `paddr`, `xid`) sample before deciding which mattered. The cost of recording is small; the cost of discovering later that a key field was never captured is large.

Chapter 11

Support, Readability, and the Fallback

A source the product makes a correctness claim on is *supported*. A source that can merely be opened and decoded is *readable*. This chapter develops the vocabulary, the current allowlist, and the fallback path the product runs when the boundary refuses a source.

11.1 Four Properties

Four properties characterise a source’s relationship to the parser. The order matters; each builds on the last.

Readable. Raw bytes can be opened and sampled. A device file exists and `read()` returns data.

Parsable. Checkpoint and root discovery succeed; raw traversal runs without aborting.

Validatable. The parser’s output can be compared to a named oracle for the requested mode. A namespace can be diffed against a POSIX traversal; a logical size against `st_size`.

Supported. The source is in the product allowlist for the requested mode. Validation has been performed; results were correct; the source class belongs in production.

The first three are technical. The fourth is editorial: the project’s decision about which sources are stable enough to make a correctness claim on. A source can be readable, parsable, and validatable in one moment and still not be supported because the moment is not durable.

A mounted live volume is readable. It may be parsable on a still moment. It may be validatable against a momentary POSIX snapshot. It is not supported, because the validation is not reproducible: the next moment moves the visible checkpoint, the next POSIX snapshot disagrees with the parser snapshot, and no row can be attributed to one moment. The general rule: support a source class only when the validation recipe is reproducible.

11.2 The Current Allowlist

The raw-mode allowlist is intentionally short.

- Detached, unencrypted, image-backed APFS containers. The `.dmg` layer fixes the visible state at detach time.

- Caller-supplied raw APFS container devices under `/dev/disk*` or `/dev/rdisk*` where the caller has committed that the checkpoint will not advance.
- Explicitly stable APFS sources — a tightly controlled lab volume where the operator pins the state.

What is not on the list: live mounted startup disks, FileVault volumes the user is using, multi-device or Fusion containers, sealed system volumes, snapshots opened through the operating system’s snapshot framework, and any container whose `omap_phys.flags` indicate an encryption operation in flight.

The allowlist will grow only after a paired oracle and a new probe clear the corresponding entry in the support matrix.

11.3 Live-Volume Raw Mode (EX-28)

A live mounted volume’s `/dev/diskNsM` device node is not on the allowlist: a live mounted volume’s metadata advances during the scan window and SR-014 checkpoint selection has no way to pin the visible state.

EX-28 asks the narrower question: with root privileges, does the existing raw parser produce a self-consistent shape on `/dev/diskNsM`, and does that shape agree with what the fallback walker emits from the same mount point in the same scan window?

The harness lands as a Rust integration test in `crates/apfs-fastindex/tests/ex28_live_parity.rs`, gated on the environment variables `APFS_FASTINDEX_EX28_LIVE_DEVICE` and `APFS_FASTINDEX_EX28_MOUNT_POINT`. Without those, the test is a clean no-op and `cargo test -release` skips it. With them set and the test run as root, the harness:

1. Tries three successive raw scans of the device. Each must publish a `selected_checkpoint` (no SR-014 fail-closure under live concurrent writes), with pairwise symmetric difference of their `NamespaceEntry` sets ≤ 200 paths.
2. Runs one fallback-walker scan of the mount point and compares it against the raw scan; symmetric difference must be ≤ 1000 paths to absorb the raw-can-read-paths-the-fallback-cannot asymmetry (TCC, sealed system volume reads).

The comparator behind both checks is `apfs_fastindex::parity::compare_namespace_shapes`, which returns a structured `ShapeDiff` with `only_in_left`, `only_in_right`, and per-field mismatches. It is exposed as a public crate surface so any future tooling can use it.

Observation (2026-05-20). On the project owner’s Apple silicon host, macOS refused to open `/dev/disk3s1` for raw reads, returning `EPERM` on the first `read(2)` of block zero even under `sudo`. The harness classifies this as `LiveScanOutcome::BlockedByKernel` and records the `live_raw_blocked_by_kernel` verdict. The kernel’s storage-system security policy refuses raw block access to the live boot data partition independently of POSIX permissions; SIP and the sealed-system-volume seal both contribute. Neither the successive-scan stability nor the raw-vs-fallback parity assertions could be evaluated because the raw backend never opened.

Product implication. The “Scan as Administrator” shell action does not graduate to raw mode on a stock Apple silicon macOS host. The privileged-subprocess shape still serves a purpose — unlocking TCC-restricted user-data paths the fallback walker hits `EACCES` on — but it stays on the fallback walker even under root. The raw fast path is unchanged in its original scope (detached `.dmg` images and caller-pinned raw devices on a quiesced volume). EX-28’s contribution

to the codebase is the reusable shape-parity comparator (`apfs_fastindex::parity`) and the env-gated integration harness, both rerunnable on future hosts or future macOS versions without code changes.

The live-disk row of the support matrix: *not supported on stock macOS; raw read returns `EPERM` under `sudo`; the fallback walker remains authoritative, including the privileged-subprocess invocation that unlocks TCC-restricted user-data paths.*

11.4 Local-Snapshot Enumeration (EX-29)

The product surfaces local snapshots as a status-bar row because their bytes are part of `df`’s “used” total and the volume’s Finder-reported capacity, but the scanner walks only the live volume’s fs-tree; without a snapshot row, those bytes look missing.

EX-29’s deliverable is enumeration: a count of user-visible local snapshots, with names. Reclaimable bytes are out of scope on stock Apple silicon. There is no public read-only oracle for per-snapshot reclaimable bytes (`tmutil thinlocalsnapshots` reports the number, but it deletes the snapshot in the process), and EX-28’s verdict rules out the raw-extent-set-diff path.

Enumeration is unprivileged. The `snapshots` module shells out to `tmutil listlocalsnapshots <mount>` and `diskutil apfs listSnapshots <mount>`, parses each output, and classifies. SR-020’s sealed-system filter excludes `com.apple.os.update-*` (the boot snapshot the user cannot delete).

Observation (2026-05-20). The project owner’s host has zero user-visible TM local snapshots and one sealed-system OS-update snapshot on `disk3s1s1`. The classifier returned `NoUserSnapshots{sealed_system_excluded_count = 1}`. EX-23 found the same shape five days earlier; on a single-user development host without Time Machine configured this is the expected steady state.

The integration harness in `tests/ex29_snapshot Enumeration.rs` mirrors EX-28’s pattern: an unconditional unprivileged test runs the classifier; a gated test exercises a raw extent diff between a mounted snapshot’s device node and the live volume via `parity::compare_namespace_shapes`. The gated test reuses EX-28’s `LiveScanOutcome` classifier, so an `EPERM` on the snapshot device is recorded rather than failing the test.

Support matrix row: *enumeration validated; reclaimable bytes unclaimed.*

11.5 Fail-Closed Triggers

A source outside the allowlist fails closed before any row is emitted. The categorical triggers, with the chapter that develops each:

Ambiguous or malformed checkpoint state (chapter 3).

Non-contiguous checkpoint descriptor layout (chapter 3).

Unsupported incompatible feature bits in the container or volume (chapter 3, chapter 5).

OMAP-open or OMAP-lookup encryption flags (chapter 4).

Unknown bits in any `flags` field the parser inspects (chapter 4, chapter 7).

Encryption keys or hardware-backed key requirements the parser cannot reach (chapter 4).

Fusion or multi-device storage requirements (chapter 3).

Requested merged-root or boot-root semantics (chapter 13).

Snapshot or sealed-volume semantics outside the target mode (chapter 13).

No validated corpus for the source class.

Each trigger is a typed error with a substring naming the rule. The caller dispatches on the error rather than retrying.

11.6 The Fallback

When the parser refuses a source, the fallback path emits the same row shape through supported operating-system APIs.

Observation. A POSIX-traversal fallback in both Python (`fallback_traversal.py`) and Rust (`fallback.rs`) emits `NamespaceEntry` and `DirectoryAggregate` rows that, on the proof fixture, match the raw-mode output of the same volume exactly. Same entry count, same paths, same `entry_kind`, same `logical_size`, same `symlink_target`, same per-directory aggregates.

The fallback walker uses `os.walk` plus `os.lstat` plus `os.readlink` in the Python version, and the equivalent `read_dir` plus `symlink_metadata` plus `read_link` in the Rust version. Both paths apply the SR-017 precedence rule at the trivial step: `st_size` for regular files, `readlink` byte length for symlinks. Both apply the unique-inode aggregate policy from chapter 8.

The row contract is mode-independent: the caller does not need to know which backend produced the row. The field set is identical.

The fallback is not slow. Chapter 12 records 313 k entries per second on warm directory trees with four workers ($2.47\times$ the post-micro-optimisation single-threaded throughput, $1.82\times$ the pre-perf baseline). On a cold whole-machine scan the disk is the bottleneck, not the parser.

11.7 Resilience In The Fallback

A whole-machine scan as a normal user encounters paths that the user does not have permission to read: sandboxed application data, sealed-system directories, daemon-owned caches. The fallback walker must continue across these without aborting and without producing silently incomplete totals.

Observation. Per-entry `EACCES`, `EPERM`, and `ENOENT` are recorded in `parser_output.walk_skips` with a reason; the walk continues. The same log carries the `firmklink_overlay_source` and `duplicate_dir_inode` skip kinds introduced by the dedup pass below. A normal-user / scan on the host the project tested completes with roughly five hundred recorded skips (the exact count varies with the host's app sandbox profile and the firmklink set) and reports them in the output document. The caller can render the skip list as a diagnostic without it affecting the row stream.

The walker also skips top-level system directories that exist only as APFS-internal artefacts on the boot disk (`.fsevents`, the system snapshot mountpoints) by name, rather than letting the operating system raise an error on them. That skip is documented in the source so it can be revisited as support broadens.

11.8 Parallel Fallback

Observation. The fallback walker accepts a `-threads N` flag. The default is `min(hw.physicalcpu, 4)`; the lower bound is 1, the upper bound clamped to 4. Per-worker `BulkReader` owns its own

64 KiB `getattrlistbulk` buffer; a shared work queue distributes directories; per-worker entry vectors are concatenated and re-sorted on the main thread.

The default’s ceiling at 4 is not arbitrary. The single APFS container holds a B-tree contention point that pre-Mojave was described as a global mutex (Szorc 2018) and that the Apple Developer Forums position (Apple DTS, 2025) describes as having been largely addressed in macOS 10.14 while leaving a residual per-container cost. Empirical measurement on Apple silicon (EX-25, chapter 12) shows the throughput envelope peaking at $T = 4$ with $2.47\times$ speedup over single-threaded, then *regressing* at $T = 8$ and falling further at $T = 14$. At $T = 8$ the total system CPU is $4\times$ the $T = 1$ value while the throughput is only $1.94\times$; at $T = 14$ system CPU is $9.3\times$ for $1.38\times$ throughput — the kernel pays super-linearly more work to deliver sub-linearly more answers. The 4-thread ceiling stays clear of that regime by construction.

The parallel walker preserves the serial walker’s contract by shape, asserted by a synthetic-tree regression test (`parallel_walker_matches_serial_shape`): identical entries, aggregates, and skip lists; the only difference is the order in which directories are visited, which the final sort normalises.

Live progress events flow through a dedicated sampling thread inside the parallel scheduler at a 250 ms cadence (matching the serial path), so the CLI’s `--progress` stream and any user-space consumer of it can tick smoothly on a whole-machine scan that is otherwise silent for tens of seconds. The in-process FFI consumed by the native shell (chapter 13) exposes the same stream through `apfs_scan_directory_with_progress`, whose callback the sampling thread invokes from the library side. The SwiftUI shell hands in a closure that updates an `@Published ProgressSnapshot` on the main actor; the determinate progress island follows it. One source of truth, two delivery surfaces.

11.9 Firmlink Overlay Dedup

Observation. On macOS the user-visible startup root is a join of the System volume and the Data volume. Specific paths — `/Users`, `/Applications`, `/Library`, `/private`, `/usr/local`, and a few others — are exposed both at their canonical location *and* at `/System/Volumes/Data/<source>` via firmlinks. The same underlying inode is visible through two paths. A naive recursive walker descends both and double-counts everything inside the overlay.

The fallback walker, when the root is `/`, reads `/usr/share/firmlinks` at scan-start and builds a `HashSet` of the `/System/Volumes/Data/<source>` overlay paths. A descent into any such path is refused with a `firmlink_overlay_source` skip; the canonical path remains the one that contributes to the namespace and aggregates.

A second layer of safety, a per-scan `Mutex<HashSet<(dev, ino)>`, catches any directory inode reached through a path the firmlink overlay set did not cover. A second observation of the same inode through a different path is recorded as a `duplicate_dir_inode` skip and the walker does not recurse.

The effect on whole-machine scans is large. Before the dedup, a `/` scan on the test host reported 5,250,000 entries; after, 3,060,000 entries with seventeen `firmlink_overlay_source` skips (one per existing firmlink target) and zero `duplicate_dir_inode` skips (the overlay layer caught everything). The forty-two per cent that disappeared were not real files: they were the same files counted twice through two paths.

11.10 Auto-Dispatch

The CLI binary `apfs-fastindex-scan` picks the backend automatically based on the source path's shape.

- A `.dmg` path or a `/dev/disk*` / `/dev/rdisk*` device path is dispatched to raw mode.
- A directory path is dispatched to fallback mode.
- The caller can override with `-mode raw`, `-mode fallback`, or `-mode auto`.

Auto-dispatch makes the support boundary usable as a product surface. The binary handles the dispatch and reports through `correctness_claim` and `not_claimed` which mode ran.

11.11 Why The Boundary Is Permanent

As the allowlist grows (chapter 13 tracks the open candidates), each new source class gets its own line in the support matrix: own allowlist row, own oracle, own probe. The boundary moves only when there is evidence to move it.

Part VI

Engineering

Chapter 12

Measurement

Performance work follows one rule: measure before optimising; record what was measured; report only numbers whose source class is unambiguous.

This chapter records five passes: the standing baseline; a user-space micro-optimisation pass; the parallel-walker pass that followed from the performance-lever map; the structural-density pass against the post-FFI workload; and the production-shape baseline that anchors the next phase of work.

12.1 The Standing Baseline

Cold cache, normal user, release build, single host.

target	backend	entries	wall (s)	entries/s	user / sys CPU
proof fixture	raw	7	0.23	30	13 ms / 15 ms
apfs-fastindex repo	fallback (std)	9 124	0.07	130 734	15 ms / 48 ms
/Applications	fallback (std)	163 667	1.28	127 513	410 ms / 728 ms
/Applications	fallback (bulk)	163 667	1.04	157 155	347 ms / 608 ms
/Users	fallback (bulk)	1 304 073	26.65	48 933	2.92 s / 6.62 s
/ (whole machine)	fallback (std)	5 251 546	129.6	40 510	16.3 s / 47.8 s
/ (whole machine)	fallback (bulk)	5 260 624	108.7	48 380	14.85 s / 29.80 s

Reading the standing baseline:

The proof-fixture raw number is dominated by setup. Most of the 0.23 s is `hdiutil attach`; the actual seven-entry walk is sub-millisecond. The raw-mode throughput on a meaningful real volume is not yet measured because the project does not carry a large detached APFS sample. The number is honest but unrepresentative.

Fallback throughput sits at roughly 130 thousand entries per second on warm trees with `lstat` and roughly 157 thousand with `getattrlistbulk`. The improvement on `/Applications` is about 24 per cent throughput. On the cold / scan the improvement is smaller: 16 per cent wall, because the disk is the bottleneck on a cold-cache run.

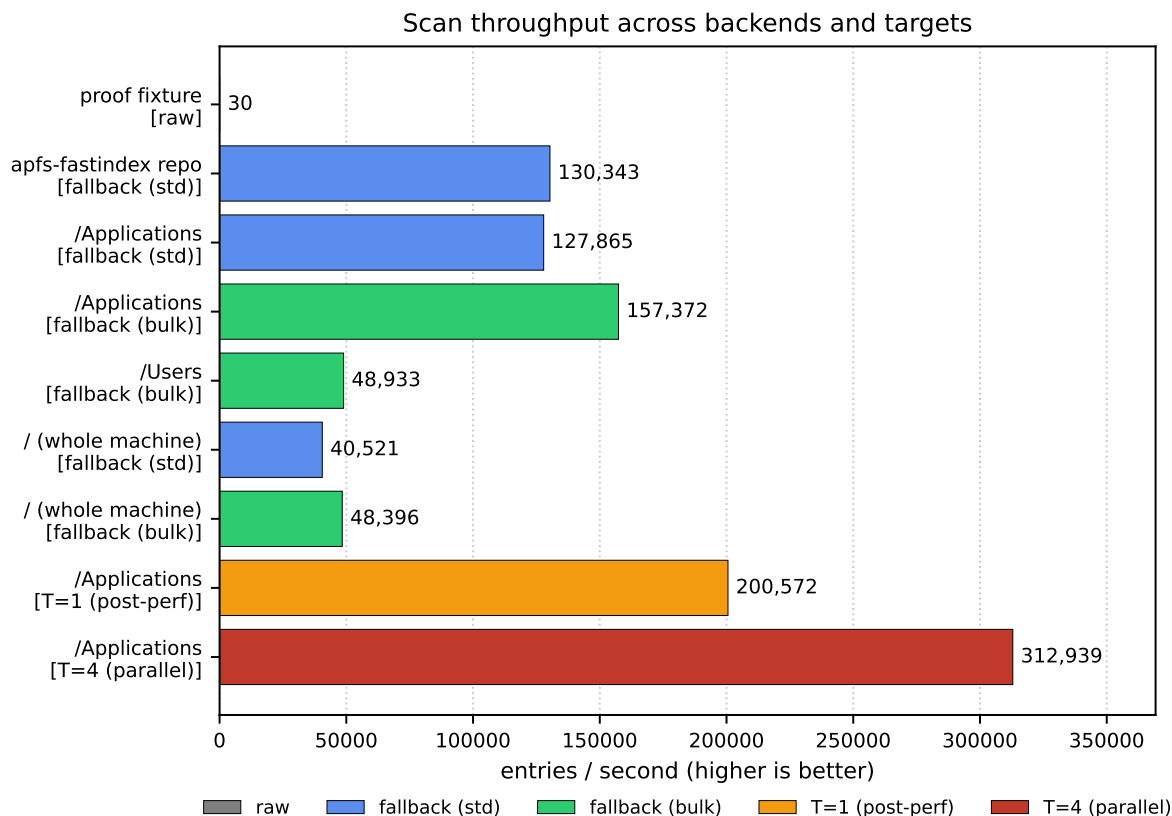


Figure 12.1: Entries per second across the measurement rows. The proof fixture is dominated by `hdiutil attach` setup; the fallback rows cluster from forty thousand to over three hundred thousand entries per second depending on backend and tree shape. The orange row is the standing single-threaded fallback after the micro-optimisation pass below; the red row is the parallel walker.

The cleaner perf signal is system CPU. On the cold `/` scan, `getattrlistbulk` drops system CPU from 48 s to 30 s — a 38 per cent reduction. Batching cuts the syscall count, and the kernel does less work per entry. The wall-time savings compound on warm-cache runs where the disk is not the constraint.

12.2 The Micro-Optimisation Pass

The standing baseline left fallback throughput on `/Applications` at roughly 172 thousand entries per second end-to-end (single-threaded, post-baseline) with system CPU dominating the budget. A user-space pass against per-entry allocations and aggregate construction landed four coupled changes:

- `BulkReader` owns one 64 KiB `getattrlistbulk` buffer and one output `Vec<BulkEntry>` across the whole walk, replacing the fresh `vec![0u8; 65_536]` the per-directory `read_directory_bulk` used to allocate.
- `build_aggregates` switched from `BTreeMap<String, BTreeMap<u64, _>` keyed by owned

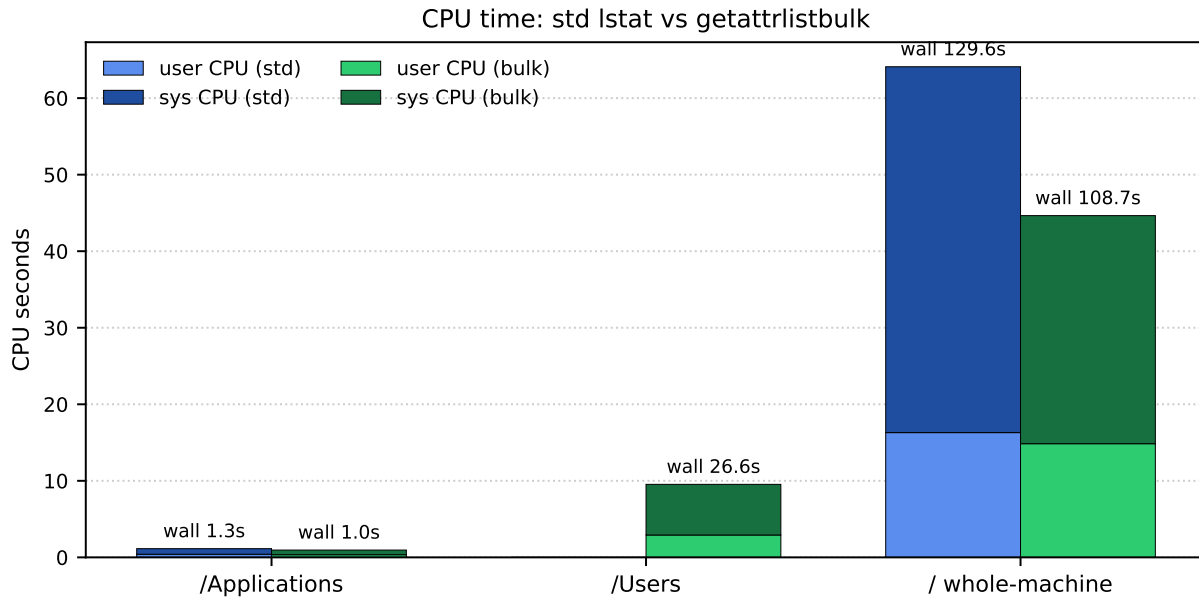


Figure 12.2: User-plus-system CPU per row, comparing ordinary `lstat` against `getattrlistbulk`. The wall-time improvement is real but modest; the system-CPU drop is the cleaner signal that bulk fetch is doing less per-entry work in the kernel.

ancestor paths to `HashMap<&str, HashMap<u64, _>` keyed by slices borrowed from each entry’s path `String`. The old shape walked ancestors as a `Vec<String>` per file — on a five-million-row / scan, twenty-five million owned-`String` allocations. The rewrite walks parents inline with `rfind('/')` and slices; the final sorted emission converts to owned `String` exactly once.

- `EntryKind` gained `Copy` (it is a four-variant unit enum), eliminating pointless `.clone()` calls in the per-entry walker hot path.
- The unconditional `relative_str.clone()` per entry was collapsed: the mount-boundary branch is the only one that ever needed a second copy; the common path moves the `String` into the `NamespaceEntry` directly. Combined with the `Copy` change above, the steady-state inner loop pushes one entry with zero heap activity beyond the `String/Vec` growth itself.

target	metric	before	after	delta
apfs-fastindex repo	median wall	58.6 ms	53.9 ms	−8%
apfs-fastindex repo	user CPU	21 ms	17 ms	−19%
apfs-fastindex repo	sys CPU	32 ms	32 ms	flat
apfs-fastindex repo	throughput	323 721 ent/s	351 757 ent/s	+9%
/Applications	median wall	950 ms	816 ms	−14%
/Applications	user CPU	340 ms	227 ms	−33%
/Applications	sys CPU	535 ms	515 ms	flat
/Applications	throughput	172 283 ent/s	200 512 ent/s	+16%

The whole-machine / scan was not re-measured here — the host’s filesystem cache state would dominate any single re-run — so the medium-tree percentages are the load-bearing data points.

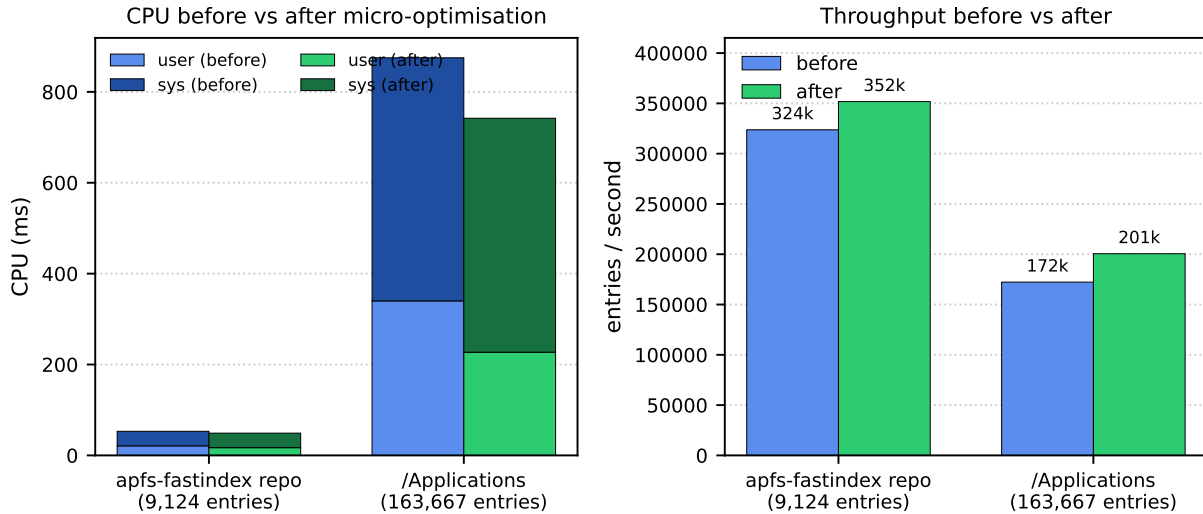


Figure 12.3: Micro-optimisation pass. The user-CPU column is the load-bearing signal: -19% on a small tree and -33% on a medium one. System CPU is flat (the syscall count did not change); only the per-directory allocator pressure is gone. The user-space savings scale with entry count because the aggregate phase dominates on larger trees.

12.3 The Performance-Lever Map

The micro-optimisation pass left the scanner at roughly 200 thousand entries per second on `/Applications` with 63 per cent of the budget in system CPU. The remaining cost was structurally kernel-side: user-space micro-opts had no further room. A source review (SR-021) cross-read four parallel research threads — xnu’s `getattrlistbulk` cost decomposition, the Spotlight alternative, an open-source-tool survey, and the `jwalk/rayon` prior art — to produce a map of which levers were structurally viable. The map flagged two candidates worth measuring (parallelism and attribute-mask shrink), two ruled-out alternatives (Spotlight, bulk buffer size), and one worth A/B testing (`fts(3)`).

Two probes ran:

EX-24: attribute-mask shrink falsification. A single-threaded microbench compared `getattrlistbulk` with three configurations on `/Applications`: drec-only (no inode-required attributes), the production mask (`ATTR_FILE_TOTALSIZE | ATTR_FILE_ALLOCSIZE`), and `fts(3)`. The drec-only and production rows landed within one per cent of each other on every dimension — wall, user CPU, system CPU, throughput — which falsifies SR-021’s hypothesis that the inode-attribute fetch path was a measurable lever on modern Apple silicon. The `fts(3)` configuration was 1.7 times slower than the production mask, contradicting Tempel’s 2019 result and cancelling the conditional follow-up. The pure kernel throughput in this microbench was 312 thousand entries per second, which gave the parallel probe a useful kernel ceiling estimate.

EX-25: parallel walker. A standalone parallel microbench swept thread counts $T \in \{1, 2, 4, 8, 14\}$ (14 being the host’s `hw.logicalcpu`). The verdict ladder required both $T = 4$ throughput $\geq 2 \times T = 1$ throughput and $T = 4$ system CPU per thread $\leq 1.2 \times T = 1$ system CPU to declare scaling validated. Both gates passed.

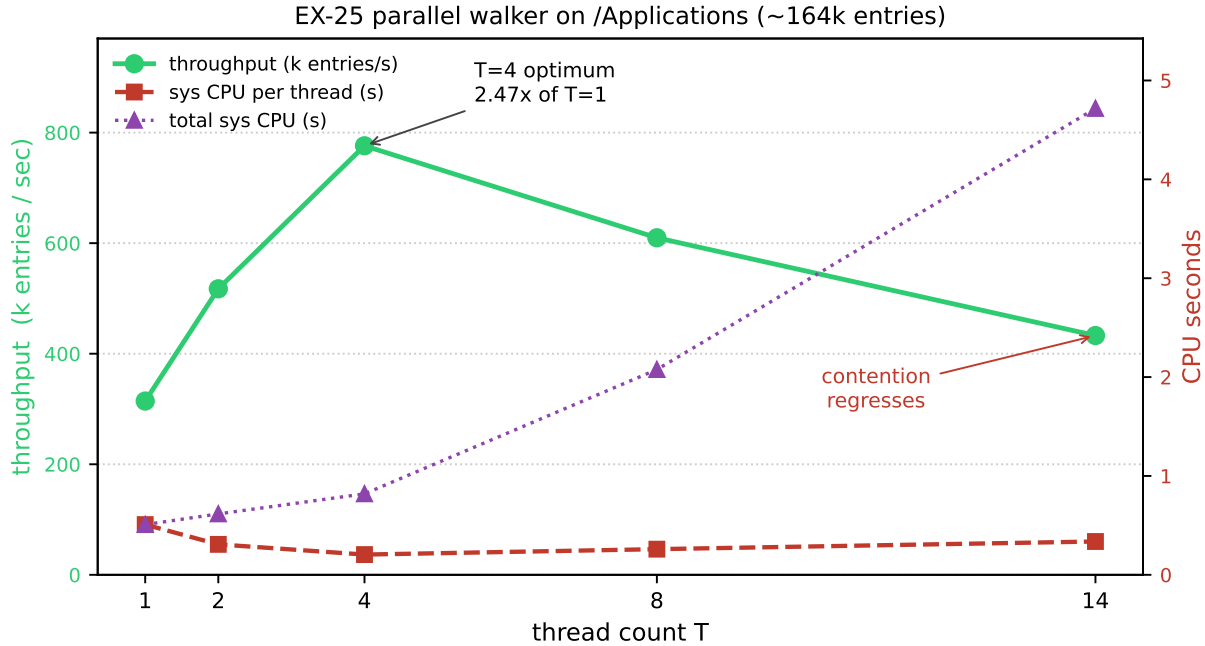


Figure 12.4: The EX-25 parallel scaling curve. Throughput rises to a peak at $T = 4$ ($2.47\times$ over $T = 1$) and then regresses; system CPU per thread stays flat through $T = 4$ and then jumps super-linearly as the APFS container contention regime fires. At $T = 8$ total system CPU is $4\times$ the $T = 1$ value for $1.94\times$ throughput; at $T = 14$ it is $9.3\times$ for $1.38\times$. The contention shape exactly reproduces the Szorc-2018 prediction on this kernel.

T	wall (s)	sys (s)	sys/T (s)	entries/s	speedup
1	0.521	0.508	0.508	314 380	1.00×
2	0.316	0.616	0.308	517 437	1.65×
4	0.211	0.819	0.205	776 196	2.47×
8	0.268	2.076	0.260	609 748	1.94×
14	0.378	4.717	0.337	432 866	1.38×

The four-thread cell is the production default. The two regression cells matter even though they are not the default: they explain *why* the default is conservative, and they tell a future maintainer how to interpret a host whose curve looks different.

12.3.1 The Parallel Slice In Product

The Rust slice landed a `-threads N` flag with default `min(hw.physicalcpu, 4)`, computed via `sysctlbyname("hw.physicalcpu")` with `std::thread::available_parallelism()` as the cross-platform fallback. The walker’s contract is preserved by shape: per-worker `BulkReader`, shared `WorkQueue` (mutex plus condvar plus outstanding counter plus an atomic done flag), per-worker entry vectors concatenated on the main thread, the existing `sort_unstable_by` plus `build_aggregates` normalising ordering. A synthetic-tree regression test (`parallel_walker_matches_serial_shape`) asserts byte-identical entries, aggregates, and walk-skip lists across the two schedulers.

End-to-end measurement on `/Applications`, same host as the previous tables, median of five runs each side:

metric	$T = 1$ (post-pass)	$T = 4$ (parallel)	delta
median wall	816 ms	523 ms	−36%
throughput	200 512 ent/s	312 879 ent/s	+56%
user CPU	227 ms	256 ms	+13% (worker setup)
sys CPU	515 ms	802 ms	+56% (kernel scales)

Cumulative `/Applications` throughput across both perf passes: $172\,283 \rightarrow 312\,879$ entries per second, +82% ($1.82\times$). The user-CPU gap between the production scanner (256 ms at $T = 4$) and the parallel microbench (about 20 ms at $T = 4$) quantifies the user-space post-processing tax — `NamespaceEntry` allocation, per-worker merge, final sort, aggregate construction. It is recorded as a candidate future optimisation; the structural-density pass below is the work that followed.

12.4 The Structural-Density Pass

The native-shell rewrite of chapter 13 introduced a new dominant workload: the FFI scan plus indexed-tree build that the SwiftUI app performs on every `Choose Folder`. The first measurements against this path on a representative real tree (`/Users/kai/Projects`, 320 000 entries, four threads, warm cache) showed where the post-rewrite cost actually sits: roughly 600 milliseconds of scan wall, 68 milliseconds of tree build, and 262 mebibytes of peak resident set. The walk-time work the previous pass had already taken; what was left was structural — the shape of the in-memory representations and the work the build phase did once the bytes were back.

A bench-driven pass (eight rounds, each backed by per-phase timings the `APFS_PHASE_TIMINGS=1` envvar surfaces and allocator-pressure counts the `examples/perf_probe` binary records) attacked that surface change by change. Each round changes one thing; each round is recorded in the commit that landed it (`git log --grep='perf('`); the cumulative shape across all eight is the table below.

metric	before	after	delta
scan wall ($T = 4$, warm)	~600 ms	~410 ms	−32% ($1.5\times$)
tree wall (warm)	68 ms	34 ms	−50% ($2\times$)
peak RSS	262 MiB	190 MiB	−28%
<code>TreeNode</code>	168 B	112 B	−33%
<code>NamespaceEntry</code>	88 B	72 B	−18%
scan-time allocations	2.23 M	2.09 M	−7%
scan-time bytes	580 MiB	330 MiB	−43%

Most of the eight rounds are mechanical: the structures the scan produces and the indexed tree reads once per scan settle into a narrower shape, with one fewer indirection per access wherever an interior allocation could be folded into a contiguous arena or a borrowed slice. Three are worth naming separately because each implicates a larger principle.

Box<str> over String for write-once fields. Both the flat `NamespaceEntry` and the indexed `TreeNode` carry strings that are built once during the walk or the index build and never mutated. The eight bytes of capacity each `String` carries against the day it might grow is wasted work on a

write-once row. Switching `path`, `symlink_target`, and `name` fields to `Box<str>` shaves twenty-four bytes per `TreeNode` across three fields and sixteen bytes per `NamespaceEntry` across two, without changing the build side: the strings already came from `into_boxed_str()` on a fresh `String` whose capacity equalled its length, so the shrink is a metadata-only operation. (Rounds 1 and 2.)

Non-cryptographic hashing for non-adversarial keys. Two of the build’s largest `HashMap`s — the `Tree::build` per-directory child map keyed by short filenames, and the `build_aggregates` outer map keyed by ancestor-path slices and inner map keyed by inode numbers — were running on `std`’s default `SipHash`, which trades collision-resistance against speed. Filenames and inode numbers are not adversarial input; the collision-resistance trade is moot. Swapping the hasher to `rustc_hash::FxBuildHasher` dropped `Tree::build` from sixty-three to forty-seven milliseconds (round 3, -25%) and `build_aggregates` from roughly 150 to 91 milliseconds (round 7, -40%) on the same input. The change is bounded to build-time maps; the read side is untouched. (Rounds 3 and 7.)

Don’t compute what the caller does not read. The `DirectoryAggregate` table the fallback produces is a flat `Vec` keyed by directory path with per-inode contribution totals. The CLI’s JSON output consumes it; the SwiftUI FFI does not (the indexed tree’s `value_logical` and `value_allocated` cover the same surface from a different angle). A boolean option on `FallbackOptions::skip_aggregates` — false by default, true on the FFI path — skips the build entirely. The change saved roughly ninety milliseconds of scan wall, 250 mebibytes of scan-time allocator pressure, and roughly thirty-four mebibytes of peak resident set on the FFI path; the CLI’s JSON shape is unchanged. (Round 8.)

The other five rounds (drop `TreeNode.path` and lazy- compute it from the parent chain; flatten per-directory `Vec<u32>` children into a single contiguous arena; shard `VisitedDirs` into sixteen mutexes keyed by a hashed (`dev`, `ino`); pre-size per-directory child vectors; `Box<str>` for the inflight child-map keys so the map and the `TreeNode.name` share one allocation) follow the same shape: identify the wasted indirection, the wasted capacity, or the wasted lock, and remove it.

Two more candidates were probed and falsified inside the pass and warrant naming so they are not relitigated. The `getattrlistbulk` buffer size is not a perf lever on this workload (every value tested from 32 KiB through 2 MiB landed within noise; 4 MiB regressed), exposed at runtime via the `APFS_BULK_BUF_KIB` envvar for any future re-ablation. And EX-24’s kernel-side falsification of the attribute-mask shrink hypothesis means the next bounded lever on the FFI path is no longer in user-space allocation or hashing — it is the walk itself, where the kernel is already doing the syscall work `getattrlistbulk` reduced.

12.5 The Production-Shape Baseline (EX-30)

The chapter’s other measurements run the walker as a library inside a benchmark loop. The shipping product invokes `apfs-fastindex-scan` as a subprocess with `--format msgpack`, optional `--progress`, and the default thread count. Process spawn, `msgpack` serialisation, `stderr` handling, and the user’s actual data shape contribute to production wall time but had never been measured end to end. EX-30 fixes the baseline against `/Users/kai`, the largest single subtree the project owner routinely scans, on the same Apple silicon host.

iteration	wall (s)	entries	entries/s
cold	9.87	1 556 792	157 760
warm 1	10.89	1 556 800	142 940
warm 2	9.62	1 556 800	161 789
warm 3	9.59	1 556 800	162 258
warm 4	9.62	1 556 800	161 766

Warm median: 9.62 s, 162 k entries/s, 1.56 M entries, 172 k directory aggregates, 392 MB msgpack payload. The cold iteration on this host could not force a true cold cache (`sudo -n purge` requires cached credentials the harness did not have) and the EX-30 record notes the degeneracy. Prior EX-25 measurements imply the walker is CPU-bound on this workload, not I/O-bound; APFS-on-NVMe rarely shows a $>1.5\times$ cold/warm gap once the FS-tree pages are in RAM.

The number anchors the R4 cache target: the warm 9.62 s is the oracle a “no-changes-since-last-scan” rescan must outperform. The acceptance criteria for the next R4 phase are (1) cold-miss does not regress more than 5% on EX-30’s baseline, (2) cache hit on an unchanged tree completes in ≤ 1.0 s, (3) partial-mutation runs interpolate linearly between (1) and (2).

12.6 What The Numbers Do Not Say

1. Raw-mode throughput on a representative source. EX-28’s `live_raw_blocked_by_kernel` verdict precludes measuring raw mode against the host’s live volumes; the project has not built a detached APFS sample large enough to saturate the raw path.
2. Encrypted-volume surfaces. The parallel walker has not been measured on encrypted APFS; per SR-021’s review of Petrenko (2020), the surviving container contention may behave differently there.

Chapter 13

Engineering Surface and Open Questions

The previous chapters develop the manual’s defended interior: one volume, one pinned state, namespace, logical, allocated, and clone-deduplicated size, fallback for sources outside the allowlist. This chapter records the engineering surface around that interior and the questions that remain open.

13.1 The Native Shell

The product surface is a SwiftUI app whose visualisation is laid out by Rust and drawn by Core Graphics. The scanner is not a subprocess; the Rust crate is linked into the app process as a static library, and every call from Swift into the indexer crosses a `cbindgen`-generated C ABI. There is no JSON serialisation, no temp file, no WebKit, no JavaScript.

13.1.1 The FFI Boundary

The crate’s C ABI is a small surface of opaque-handle constructors, accessors, and destructors:

`apfs_scan_directory(_with_progress)` returns an `ApfsScan*`. The handle owns the `FallbackScanOutput` (entries, walk-skips, `correctness_claim`, source descriptor), the indexed tree built from it, and a lazy path-string cache.

`apfs_layout_new(scan, subtree, depth, metric, dims)` returns an `ApfsLayout*` whose flat `ApfsCell` array Swift reads as `UnsafeBufferPointer<ApfsCell>`: no per-cell call into Rust during paint, no array copy across the language boundary.

`apfs_scan_ext_summary_new` returns an `ApfsExtSummary*` with the per-extension aggregate the right-hand panel reads.

`apfs_scan_search_names` returns the node-indices of substring matches sorted by the active metric.

Bulk data crosses the boundary as `(*const Item, usize)` pairs Swift consumes via `UnsafeBufferPointer<Item>` — no copy, no per-element call into Rust. Per-handle `_free` functions release every allocation `_new` produced. The header regenerates from `src/ffi.rs` on every change via `cbindgen`, so the C declarations cannot drift from the Rust `#[no_mangle]` extern "C" surface.

13.1.2 Division Of Labour

Rust owns: the fallback walker, the firmlink dedup, the indexed tree (one `TreeNode` per directory and per leaf, parent and children indices, pre-computed `value_logical` / `value_allocated` / `value_real` / `item_count` aggregates), the squarified treemap layout (a direct port of `d3-hierarchy`’s `treemapSquarify`), the 64×64 hit-grid, the per-extension aggregator behind the right-hand panel, the substring-search index, and the colour palette that decides a leaf’s `fill_rgb`.

Swift owns: the SwiftUI shell (toolbar, breadcrumb, folder-tree side panel, ext-list side panel, search field, status bar, depth stepper, metric picker), the AppKit treemap view (one `NSView` whose `draw(_:)` walks the cell array grouped by colour to amortise `setFillColor` calls), the right-click menu plumbing onto `NSWorkspace` (`Open`, `Reveal in Finder`, `Copy Path`, `Move to Trash`), the sticky admin-mode flag and privileged-subprocess invocation, the snapshot enumeration status pill, and the Sparkle updater.

13.1.3 Treemap Layout

The layout is the standard squarified treemap of Bruls, Huijzen, and van Wijk (2000), in the variant `d3-hierarchy` implements. Children are sorted descending by the active metric; greedy row-packing on the shorter side of the current rect adds children to a row until the worst-case aspect ratio would worsen, then lays the row out by dice (horizontal) or slice (vertical) and recurses on the remaining strip. The Rust port preserves the `d3` expressions for α and β ; for the same input (`node_index`, `value`) array and viewport the two implementations produce the same rectangles.

A 64×64 axis-aligned spatial hash sits over the cell array. Each cell is inserted into every bucket it overlaps; each bucket is sorted depth-descending so the first containing match a hit-test walk sees is the deepest cell at that point. Mouse-move becomes a few-cell comparison rather than a walk through the hundred-thousand-cell layout for a `/-` scan view.

When the metric picker is on `Allocated` (or `Real`) and a leaf’s `value_allocated` (or `value_real`) is `None`, the cell’s `fill_rgb` is the unclaimed grey rather than a colour from the extension palette. The fail-closed register surfaces on the picture.

13.1.4 Substring Search (EX-33)

The shell ships a substring search over already-enumerated names. The C ABI exposes `apfs_scan_search_names(scan, needle_ptr, needle_len, limit)`, which returns a sorted slice of matching node indices. The implementation is the result of a two-round perf study.

Round 1 (pre-EX-33). The naive `Vec<String>`-of-pre-lowercased-names plus `str::contains` brought per-keystroke latency down from multi-second to roughly 30–55 ms on the 1.56 M-entry `/Users/kai` scan — a $100\times$ win, still $10\text{--}20\times$ off the memory-bandwidth floor for 50 MB of name bytes.

Round 2 (EX-33). Two compounding costs remained: the `Vec<String>` layout put each name behind its own heap allocation (1.5 M random-pointer reads are $10\text{--}20\times$ slower than sequential bytes); and `str::contains` built its searcher state once per call (1.5 M searcher constructions for the same needle). The fix replaced the storage with a flat `Vec<u8>` plus a cumulative `Vec<u32>` offset table, and built one `memchr::memmem::Finder` per search reused across every haystack.

query	before	after	speedup
".txt"	32.5 ms	10.1 ms	3.21×
"Library"	41.7 ms	11.8 ms	3.53×
"node_modules"	53.2 ms	9.5 ms	5.59×
"com.apple.developer"	34.6 ms	5.3 ms	6.53×
zero-match (memory-bandwidth floor)	27.5 ms	2.3 ms	12.0×

Typical 4–12-character queries now median 9–12 ms on the 1.5 M entry tree — comfortably inside the “feels live” threshold (100 ms) and within range of 60-Hz frame-rate (16 ms) for hot-cache repeats. The shorter-needle high-match cases see modest gains because their cost is dominated by ancestor walking on millions of matches; long-needle queries hit the memory-bandwidth floor.

The search is substring only; it does not perform an APFS-hash lookup (chapter 9) and inherits no risk from the missing hash.

13.1.5 What Is Not In The Shell

- Snapshot-content traversal: the SR-020/EX-23 oracle requires the privileged `mount_apfs -s` step; EX-29 confirmed reclaimable-bytes accounting is blocked by EX-28 on the same host class.
- Live raw mode: EX-28 records the `live_raw_blocked_by_kernel` verdict. The privileged-subprocess shape still runs the fallback walker to unlock TCC-restricted paths, but does not graduate to raw mode.
- Code signing and notarisation.

13.2 Boot-Root Semantics (EX-31)

Gate 3 of the roadmap asked: define the semantic target for a Finder-visible scan, build an oracle, compare raw System and Data outputs against the user-visible tree, decide whether the mode is API-only or raw-capable.

EX-28 foreclosed the raw comparison (the raw read returns `EPERM` on the live boot data partition). EX-31 fills in the rest: the only product mode the app can deliver on a typical macOS install is Finder-visible startup root via POSIX traversal, and that is exactly what the existing fallback walker already does. POSIX traversal transparently follows firmlinks; the user scanning `/` gets one tree matching what Finder shows — `/Users` and `/System/Library` both appear in expected positions even though `/Users` lives on the data volume and `/System/Library` on the sealed-system snapshot.

Observation. On the project owner’s host every top-level path under `/` reports the same `st_dev` from `stat()` / `fstatat()` — the kernel exposes the system volume’s dev for every path under `/` regardless of which volume the bytes physically live on. The `st_dev`-based “is this a different filesystem?” heuristic carries no firmlink information; the firmlink table at `/usr/share/firmlinks` is the only authoritative source. The walker reads it at scan-start for the dedup pass of chapter 11; nothing else is needed.

Verdict: `api_only_already_correct`. A future polish would surface a more specific mode label (`fallback_finder_visible` rather than bare `fallback`) so consumers can distinguish a merged-Finder scan from a single-external-volume scan; the current envelope does not. Slotted for a follow-up.

13.3 Encryption (EX-32 Scratchpad)

FileVault and per-file encryption sit at the most opaque corner of APFS. The on-disk structures are named in the reference (`apfs_keybag_loc`, `apfs_kb_locker_t`, `apfs_keybag_entry_t`, `crypto_state_t`, `wrapped_meta_crypto_state_t`, `cp_key_class_t`, `INCOMPAT_ENC_ROLLED`), but field-level semantics, the cryptographic primitives at each layer, and the key-derivation chain are described only at a behavioural level. Bit layouts, KDF input strings, per-block IV derivation, and the Secure-Enclave relationship come from third-party reverse-engineering (`apfs-fuse`, `linux-apfs-rw`, FileVault-to-APFS transition writeups).

EX-32 is a research scratchpad, not a feature. The current crate exposes `crypto_meta` as a non-production module whose public symbols decode keybag and per-file crypto-state shapes against synthetic fixtures. The deliverable is documentation; no decryption path is implemented and none is planned without an oracle. The encryption row of the support matrix stays at *hard-stop at OMAP open*.

13.4 Open Questions

13.4.1 Exclusive and Shared Bytes

The Real metric of chapter 8 charges each inode its share of clone-deduplicated bytes via the extent-reference tree. *Exclusive* bytes — bytes only this inode owns, after accounting for clones and snapshots — additionally requires the snapshot-metadata tree to be opened and joined against per-inode extents. *Shared* bytes are the complement. Neither has been validated against a public oracle; both are open.

13.4.2 Snapshot-Assisted Scanning

The mechanism for honest scanning of a live machine. The walker already accepts a `--snapshot <mountpoint>` flag that runs the fallback against an already-mounted snapshot directory; the missing piece is the shape-parity oracle.

Privilege boundary (SR-020). `fs_snapshot_create(2)` requires root and the `com.apple.developer.vfs.snapshot` entitlement. The same gate applies to delete, rename, mount, and revert. `mount_apfs -s` also requires root. `tmutil localsnapshot` is the only unprivileged create path, available only on Time-Machine-included volumes.

Reproducer. The probe is re-runnable; EX-23's clean dev workstation found no user-visible TM snapshots and emitted `blocked_no_snapshots_at_all`. The privileged rerun reads:

```

1 tmutil localsnapshot /
2 tmutil listlocalsnapshots /
3 sudo mkdir -p /tmp/apfsfi-ex23-snap
4 sudo mount_apfs -s <snapshot-name> / /tmp/apfsfi-ex23-snap
5 python3 docs/research/experiments/EX-23-snapshot-shape-parity/artifacts/probe_ex23.py
6 sudo umount /tmp/apfsfi-ex23-snap

```

A positive verdict would unlock the snapshot-mount cell of the support matrix from `not_claimed` to `fallback_supported`.

13.4.3 Persistent Incremental Cache

The R4 v1 cache (chapter 10) ships on stable trees and degenerates on home-directory-scale churn. The R4 v2 promise — walker-emitted signatures plus per-subtree caching — needs a probe against the EX-30 baseline. Acceptance: 9.62 s warm baseline, ≤ 1.0 s unchanged-tree rescan, linear interpolation under partial mutation.

13.4.4 Encrypted Surfaces

FileVault, per-file class keys, hardware-tied key wrapping. The parser fails closed on every encryption flag (chapter 4); opening support requires deciding which key-access classes the reader is willing to use. EX-32 is the documentation scratchpad; no feature follows from it without an oracle.

13.4.5 Live Boot Disk Raw Mode

EX-28's verdict `live_raw_blocked_by_kernel` stands on stock Apple silicon under SIP. The harness remains re-runnable on future hosts or future macOS versions with different security policies without code changes.

Glossary

The vocabulary the manual uses, in alphabetical order. Terms that appear in code identifiers carry the typewriter font of the identifier as well as the prose definition.

Allocated size. Bytes the filesystem has assigned to a file’s content (`st_blocks * 512` oracle). Validated by EX-26 for every shape the macOS write path produces: ordinary, clone, hard-link, sparse, decmpfs (xattr-stream-stored and resource-fork-stored), symlink, and directory. Emitted as `allocated_size: Option<u64>`; `None` indicates a deliberately-not-claimed row.

alloted_size. The `j_dstream_t.alloted_size` on-disk field. The source for the allocated-size column on the case classes that emit a value. On macOS-produced volumes the field equals `round_up(ds_size, blocksize)`; on sparse files this overstates the actual allocation by the sparse-bytes hint.

APSB. APFS volume superblock (`apfs_superblock_t`). Reached through the container OMAP at the scan XID; carries the volume OMAP, the FS-tree root, feature masks, role, UUID, and statistics.

Block zero. The fixed locator block at the start of an APFS container. Carries enough metadata to find the checkpoint descriptor area but is not itself authoritative for the live container state.

ApfsCell. The flat, `#[repr(C)]` record the Rust renderer hands across the FFI as a contiguous array. Holds the cell’s rectangle, depth, the index of the tree node it represents, a flag bitset (`is_dir`, `is_symlink`, label-strip reserved), and a precomputed `fill_rgb`. Read by Swift as an `UnsafeBufferPointer<ApfsCell>` with no array copy.

ApfsLayout. The Rust handle that owns the laid-out `ApfsCell` array and the spatial-hash index used for hit-testing. Constructed by `apfs_layout_new`, freed by `apfs_layout_free`. One `ApfsLayout` represents one (subtree-root, depth, metric, width×height) configuration.

ApfsScan. The opaque handle the C ABI returns from `apfs_scan_directory` (or the `_with_progress` variant, which adds a sampled callback at the same 250 ms cadence as the CLI’s `--progress` stream and carries a cumulative `bytes` field so the determinate progress island has a denominator). Owns the `FallbackScanOutput` (entries, walk-skips, `correctness_claim`, source descriptor), the indexed tree, and a path-string cache the `apfs_scan_node_path` accessor populates on first query (chapter 12, structural-density pass round 4). Released by `apfs_scan_free`.

BulkReader. The Rust struct that owns a reusable 64 KiB `getattrlistbulk` buffer and one output `Vec<BulkEntry>` across the whole walk. The micro-optimisation pass introduced it to replace the per-directory fresh allocation.

--cache. CLI flag (off by default) that enables the v1 persistent fallback cache (chapter 10). The cache lives under `~/Library/Caches/apfs-fastindex/` and is keyed by a probe-derived signature of the scanned directory tree. Useful on stable trees (5–50× speedup); not recommended for active home directories (mtime churn invalidates every signature; probe overhead exceeds the walk on 172 k-directory trees). EX-30 records the per-target numbers.

cbindgen. The Rust crate that walks `src/ffi.rs` and emits the C header Swift imports. Run by the `build.rs` on every change so the C declarations cannot drift from the Rust `#[no_mangle] extern "C"` surface. The generated header lives in `target/<profile>/include/apfs-fastindex.h`.

Checkpoint descriptor area. A contiguous run of blocks that hold the rotating ring of container superblocks and their associated checkpoint maps. The authoritative live NXSB lives here.

Checkpoint map. A block carrying ephemeral-OID mappings for the selected checkpoint. Walked in order from `nx_xp_desc_index`; the last block carries the `CHECKPOINT_MAP_LAST` flag.

decmpfs. The Apple compression xattr family. Small compressed files store their data inline in `com.apple.decmpfs` with no `j_dstream_t`; larger files use `com.apple.ResourceFork`. The parser fails closed on the allocated-size column for any inode whose only size source is decmpfs, because no public reader has been oracle-validated against the macOS `st_blocks * 512` for the resource-fork path.

Directory aggregate. A per-directory total expressed under the unique-inode logical-sum policy: each inode contributes its logical size at most once per aggregate root, regardless of how many hard-linked paths inside that root name it. Allocated-size aggregates use the same rule plus a `None`-propagation step.

DIR_REC. Directory record. The FS-tree leaf that represents a directory entry; key carries the parent inode’s file ID and the name; value carries the child file ID and the POSIX entry type.

Dstream. An APFS data stream. Logical and allocated size plus a default-extent reference; carried inline in the inode’s xfields as `INO_EXT_TYPE_DSTREAM` or in a separate `j_dstream_id_val_t` record.

Ephemeral object. An APFS object that lives in memory between transactions and is written into the checkpoint data area at commit. The checkpoint map identifies it by `(oid, paddr)`.

Fail closed. The project’s discipline of refusing unknown, malformed, or out-of-allowlist input with a typed reason, rather than producing a partial or best-effort result.

Fallback path. The supported-API backend that runs when raw mode rejects a source. POSIX walk plus `lsstat` plus optional `getattrlistbulk`; emits the same row shape as raw mode.

Firmlink. A macOS presentation mechanism that joins selected paths from a System volume to corresponding paths on a Data volume to produce the user-visible startup namespace. Out of scope for raw mode. The fallback walker reads `/usr/share/firmlinks` at scan-start and skips the `/System/Volumes/Data/<source>` overlay paths so the canonical path is the one that contributes to namespace and aggregates.

FsRecordDump / FsRecordRow. The Rust crate’s structured output for the FS-tree record-family dump and one decoded record respectively. Field-level equal to the independent Python decoder on the proof fixture.

FS tree. The volume-level B-tree of variable-length records keyed by (`object_id`, `type`). Holds directory entries, inodes, xattrs, sibling-link records, dstream records, and extents.

getattrlistbulk. macOS syscall that returns metadata for many directory entries per call. Used by the fallback walker for the bulk perf path. Adding inode-required attributes (size, mode, mtime) to the attribute mask used to be thought a load-bearing cost; EX-24 falsified that for modern Apple silicon.

Hit-grid. A 64×64 axis-aligned spatial hash sitting over the `ApfsCell` array. Each cell is inserted into every bucket it overlaps; each bucket is then sorted depth-descending so the first containing match a hit-test walk sees is the deepest cell at that point. Constant-time lookup turns mouse-move into a few-cell comparison even on a /-scan layout.

Hard link. A directory entry that names an inode already named by another directory entry. APFS represents the relationship through `SIBLING_LINK` and `SIBLING_MAP` records; the unique-inode aggregate policy ensures the inode's logical size is counted once per aggregate root.

Extent-reference tree. The standalone B-tree holding `phys_ext` records, one per shared physical extent. Each record carries (`paddr_first`, `length_blocks`, `refcnt`, `owning_obj_id`) and is the authority for the Real metric (chapter 8). Storage class is `OBJECT_TYPE_PHYSICAL` on detached `.dmg` images (root OID is the `paddr` directly) and may be virtual on live volumes (OMAP lookup at the scan XID).

Indexed tree. The in-memory `Tree` the FFI builds once per scan from the flat `Vec<NamespaceEntry>`. One `TreeNode` per directory and per leaf, 112 B per slot post-structural- density-pass, carrying name (as `Box<str>`), parent index, a (`children_start`, `children_count`) pair into a contiguous `children_arena` shared across the whole forest, own logical, allocated, and real sizes, and post-order-computed `value_logical`, `value_allocated`, `value_real`, and `item_count` aggregates. The path string is not stored on the node; it is lazy-computed by walking the parent chain when `apfs_scan_node_path` first asks for it, then cached on the `ApfsScan`. Random-access reads by index are constant-time; consumers need the indexed shape for the breadcrumb, the tree-list panel, the squarify recursion, and the hit-test loop.

INODE. Inode record. The FS-tree leaf that carries file/dir/symlink identity, mode, parent ID, link count, internal flags, and an xfield tail.

Logical size. User-visible byte length. The primary size metric, equal to POSIX `st_size` for regular files and to the symlink target byte length for symlinks. Distinct from allocated, exclusive, shared, and snapshot-retained sizes.

Max XID. The XID argument to an OMAP lookup. Returns the entry with the highest stored XID less than or equal to the requested value. The whole point of pinning the scan XID.

NamespaceEntry. One row of the namespace output: path, entry kind, file ID, logical size, optional allocated size, optional symlink target. Stable across raw and fallback backends.

Native shell. The SwiftUI macOS app whose visualisation is laid out by Rust and drawn by Core Graphics. The Rust crate is linked into the app process as a static library; every call from Swift into the indexer crosses a `cbindgen`-generated C ABI. No subprocess, no JSON serialisation, no WebKit. Chapter 13 documents the architecture; the repository still ships `viz/index.html` as a standalone drop-a-file-into-browser surface.

NX / NXSB. APFS container superblock (`nx_superblock_t`). The block-zero copy is the locator; the authoritative selected NXSB lives in the descriptor area at the highest valid checkpoint XID.

OID. Object identifier. A virtual address into an APFS OMAP’s name space. Not by itself a safe cache key (chapter 10).

OMAP. Object map. The B-tree that resolves virtual OIDs to physical addresses under a chosen scan XID. Container and volume each have their own.

Paddr. Physical block address. The on-disk location of an APFS object. Returned by an OMAP lookup for virtual objects; equal to `o_oid` for physical objects.

Parallel walker. The default fallback scheduler. Per-worker `BulkReader`, shared work queue, per-worker entry vectors merged on the main thread. Default thread count `min(hw.physicalcpu, 4)`, clamped to `[1, 4]`. The ceiling stays clear of the APFS container-lock regime that fires at $T > 4$ on Apple silicon.

Parsable. Source property: checkpoint and root discovery succeed; raw traversal runs. Necessary but not sufficient for “supported.”

--progress. CLI flag emitting stderr progress events at a 250 ms cadence. The parallel walker delivers events through a dedicated sampling thread so the cadence matches the serial path. The same stream is exposed on the FFI via `apfs_scan_directory_with_progress`, whose `ApfsProgressEvent` adds a cumulative logical-bytes field the SwiftUI shell uses as a determinate progress denominator.

Raw mode. The native APFS-parsing backend. Used when the source is in the raw-mode allowlist (detached `.dmg`, caller-supplied raw device, explicitly stable APFS source).

Real size. Per-inode clone-deduplicated allocated bytes (chapter 8, EX-27). Computed from the extent-reference tree’s `phys_ext` refcounts and the `j_dstream_id_val_t` record’s per-dstream refcount. Summed across all inodes on a volume, the metric equals $\sum \text{phys_ext.length} \cdot \text{block_size}$ modulo integer-division rounding of $\leq \text{refcnt} - 1$ bytes per clone-shared dstream. The fallback walker emits `real_size = allocated_size` (no refcount info from POSIX); raw mode emits `None` when the extent-reference tree did not validate.

Readable. Source property: raw bytes can be opened and sampled. Cheap. Not, on its own, evidence of correctness.

apfs_scan_search_names. FFI entry point backing the shell’s substring-search field. Takes a needle pointer plus length, returns matching node indices sorted by the active metric. Backed by a flat name buffer plus `memchr::memmem::Finder` after EX-33’s lift. Per-keystroke median latency 9–12 ms on a 1.5 M-entry tree for typical 4–12-character queries.

Scan XID. The transaction identifier the parser pins for the duration of a run. All object resolution is performed against this XID. Picked as the highest checksum-valid NXSB XID in the descriptor area.

Sibling link / sibling map. Hard-link records. `j_sibling_link_val_t` carries a hard-link’s sibling-group identifier and the linked name; `j_sibling_map_val_t` closes the mapping from sibling ID to inode.

Snapshot, APFS. A read-only view of a volume at a point in time, named in the volume’s snapshot meta tree. Sealed-system snapshots (`com.apple.os.update-*`) are out of scope. The `--snapshot <mountpoint>` flag accepts an already-mounted snapshot directory; the mount step itself is privileged (chapter 13).

Squarify. The treemap-layout algorithm of Bruls, Huijzen, and van Wijk (2000), in the variant `d3-hierarchy`’s `treemapSquarify` implements. Greedy row-packing on the shorter side of each rect; rows grow until the worst-case aspect ratio would worsen, then lay out by dice (horizontal) or slice (vertical) and recurse on the remaining strip. The Rust port preserves the `d3` expressions for `alpha` and `beta`, so the two implementations produce the same rectangles for the same input.

Sparse bytes. `INO_EXT_TYPE_SPARSE_BYTES`: the count of bytes the sparse-allocation layer has not committed to disk. An allocation hint, not a logical size. Treated as diagnostic in the logical-size precedence and as the fail-closed gate for the allocated-size rule on sparse files.

Supported. Source property: the source is in the product allowlist for the requested mode. Validation has been performed; results were correct; the source class belongs in production.

--threads N. CLI flag selecting the parallel walker’s worker count. Default `min(hw.physicalcpu, 4)`; `N = 1` forces serial mode; larger values are accepted but pay the APFS-container contention cost documented in chapter 12.

Validatable. Source property: parser output can be compared against a named oracle for the requested mode. A prerequisite of “supported” but not, by itself, sufficient.

Virtual object. An APFS object whose `obj_phys_t` storage flag indicates the object is resolvable through an OMAP. `o_oid` is independent of the block’s `paddr`; the header is validated against `omap_lookup.oid`.

walk_skips. The fallback walker’s per-entry skip log. Records `EACCES`, `EPERM`, `ENOENT`, `firmlink_overlay_source`, and `duplicate_dir_inode` reasons with the path and keeps the walk going.

XID. Transaction identifier. Monotonic per container. Used to order checkpoints (chapter 3) and to disambiguate OMAP entries (chapter 4).

Xfield. The variable-length tail of an FS-tree record that carries extended fields. One source-backed cursor rule (chapter 6); fail-closed on any malformed shape (chapter 7).

Appendix: Experiment Register

A reference index of the controlled probes the manual cites. Each entry names the question the probe answered and the verdict it produced. Full READMEs, fixture-build scripts, and saved artifacts live under `docs/research/experiments/EX-*/`.

Probe	Question and result	Verdict
EX-01	Does the latest visible NXSB on a mounted lab image stay stable under concurrent writes? Found that it does not; latest-state raw reads do not have a stable oracle on live mounted volumes.	live-state-non-stable
EX-02	What minimum FS-record taxonomy reproduces the v1 namespace plus logical-size oracle? Identified DIR_REC, INODE, dstream fields, XATTR, SIBLING_LINK, SIBLING_MAP as the v1 record families.	required-record-set
EX-03	On a detached APFS image, does the raw walker reproduce the mounted POSIX namespace plus logical size? First positive proof loop.	pinned-state-parity
EX-04	Does the EX-03 parity hold across case-insensitive and case-sensitive APFS variants? Expanded fixture corpus; parity held.	expanded-corpus-parity
EX-05	On a still-mounted lab image under write churn, does latest-state raw output match any baseline or final mounted oracle? Counterexample result: it does not. Live latest cannot be a v1 model.	live-latest-not-validatable
EX-06	Across mutations to a tracked subtree, which identity fields stay stable? OID stays stable; paddr, XID, checksum, and block hash all change. Bare-OID cache is unsafe (chapter 10).	oid-alone-unsafe
EX-07	Under exact node-identity matches across the full nine-field key, does subtree-summary reuse ever produce a wrong summary? First run found zero false reuse in the detached lab corpus. Cache idea remains alive but unproven for production.	zero-false-reuse-on-corpus
EX-08	What does the read-path support matrix look like on a safe host? Detached unencrypted image-backed APFS is supported for narrow v1. Mounted images are readable but not supported. Startup-disk raw read is blocked by privilege.	read-path-matrix-baseline
EX-09	Which size metrics belong in v1, and which require their own oracle? Defined the accounting register: logical size is v1; allocated, exclusive, shared, snapshot-retained are deferred.	accounting-register

Probe	Question and result	Verdict
EX-10	Does the native Rust checkpoint scanner reach selected NXSB, checkpoint-map, container OMAP, volume superblock decode, volume OMAP, and FS-tree root validation on the proof fixture? Yes.	native-checkpoint-scanner
EX-11	Does the checkpoint-map ring validate end-to-end with CHECKPOINT_MAP_LAST enforced? Yes, on the proof fixture and on synthetic malformed cases.	checkpoint-map-integrity
EX-12	Does the native OMAP resolver satisfy SR-006 lower-bound (oid, max_xid) semantics, with hard stops at the encryption and unknown-flag cases? Yes; cross-validated against go-apfs identitydump.	validated-omap-lookup-contract
EX-13	Does a Python raw-byte body decoder reconstruct the mounted namespace and logical-size oracle on the proof fixture? Yes, modulo candidate-scored xfield layout (later resolved by SR-015 / EX-16).	validated-native-record-body-contract
EX-14	Does a second, denser xfield-bearing fixture variant resolve the xfield layout question? Returned oracle_inconclusive: Rust aborted with a checksum mismatch at block 1031 before xfield comparison could run. Blocker tracked into EX-15.	oracle-inconclusive
EX-15	What does block 1031 actually contain, and which of (a) stale- checkpoint selection, (b) missing checkpoint-map rule, (c) Rust validation bug, or (d) malformed source is the cause? Hypothesis (c). FS-tree internal-node 8-byte values are virtual OIDs requiring volume-OMAP resolution, not paddrs. Fix landed; two regression tests added.	validated-fs-tree-internal-oid-resolution-gap
EX-16	Does the SR-015 single xfield cursor rule decode every required xfield in the proof fixture with <code>xf_used_data == sum(round_up(x_size, 8))</code> and oracle parity? Yes, 14 of 14 records.	validated-sr-015-cursor-rule
EX-17	Does the Rust body decoder hard-stop on every SR-016 per-record malformed case with a typed <code>InvalidObject</code> ? Yes; 21 unit tests, all passing.	validated-sr-016-fail-closed-unit-tests
EX-18	Does the Rust body decoder produce field-level identical output to the Python decoder on the proof fixture? Yes: 53 records, identical (<code>node_paddr</code> , <code>entry_index</code>) set, zero divergent fields.	field-level-parity
EX-19	Does the SR-017 per-inode logical-size precedence reproduce <code>st_size</code> for ordinary, sparse, clone, hard-linked, symlink, and compressed cases? Yes, 5 of 5 unique inodes; the compressed case requires <code>INODE_HAS_UNCOMPRESSED_SIZE</code> to beat the decmpfs header.	validated-sr-017-precedence
EX-20	Do Rust paths match POSIX paths byte-for-byte on both case- insensitive and case-sensitive APFS volumes? Yes; volume case and normalisation flags propagate correctly. Lookup-by-name still not claimed.	validated-sr-018-name-preservation

Probe	Question and result	Verdict
EX-21	Does a POSIX-traversal fallback emit the same <code>NamespaceEntry</code> / <code>DirectoryAggregate</code> shape as Rust raw mode on the proof fixture? Yes; Python and Rust ports both pass.	validated-fallback-skeleton
EX-22	Does the SR-019 per-inode allocated-size precedence reproduce <code>st_blocks * 512</code> for ordinary, sparse, clone, hard-link, symlink, and decmpfs cases? Three of five emit-rows match; sparse overstates by exactly the sparse-bytes hint; decmpfs is correctly fail-closed. Sparse case split into a dedicated fail-closed branch pending an EX-22b sparse corpus.	partial-validated-sr-019-alloced-size
EX-23	Does the existing fallback walker produce byte-identical rows when pointed at a live volume and a read-only snapshot of the same volume? Probe ran on a clean dev workstation, found only the sealed-system OS-update snapshot (correctly skipped), and emitted a blocked verdict with the privileged-rerun reproducer. The shape claim itself remains unclaimed pending the rerun.	blocked-no-snapshots-at-all
EX-24	For the production walker's <code>getattrlistbulk</code> attribute mask, is the SR-021 vnode-rage hypothesis a measurable lever? Drec-only and current-mask landed within 1% on every dimension; <code>fts(3)</code> was $1.7\times$ slower than the current mask. The vnode-rage cost is not load-bearing; the Phase-2 deferred-attribute refactor is de-prioritised; <code>fts</code> follow-up cancelled.	vnode-cost-marginal
EX-25	What is the parallel walker's scaling envelope on Apple silicon at $T \in \{1, 2, 4, 8, 14\}$? $T=4$ reaches $2.47\times$ $T=1$; $T=8$ regresses; $T=14$ catastrophically loses ($9.3\times$ $T=1$ system CPU for $1.38\times$ throughput). The Szorc-2018 / Apple-DTS-2025 container-lock contention regime is real and starts at $T = 2 \times$ optimum on this kernel. Production default <code>min(hw.physicalcpu, 4)</code> .	validated-parallel-scaling
EX-26	Against the post-FFI workload (<code>/Users/kai/Projects</code> , 320k entries, $T=4$, warm cache), how much of the post-rewrite budget is structural rather than kernel-bound, and which structural rounds clear which fractions of it? Eight rounds landed against <code>TreeNode</code> , <code>NamespaceEntry</code> , <code>Tree::build</code> , <code>build_aggregates</code> , <code>VisitedDirs</code> , and the CLI-only aggregates <code>Vec</code> ; net -32% scan wall, -50% tree wall, -28% peak RSS, allocator pressure roughly halved (chapter 12 § Structural-Density Pass). EX-26 also closed the SR-019 sparse + decmpfs allocated-size precedence: all sparse and decmpfs shapes match <code>st_blocks * 512</code> exactly under <code>allocated_size - sparse_bytes</code> and Σ stream-backed-xattr <code>allocated_size</code> respectively.	validated-structural-density-pass; validated-sparse-and-decmpfs

Probe	Question and result	Verdict
EX-27	For a fixture with one or more <code>cp -c</code> -created clone families, can the raw parser walk the extent-reference tree, recover per-extent refcounts, and compute clone-deduplicated per-inode bytes that sum to the on-disk allocation? Yes: <code>dstream_dedup</code> computed from <code>phys_ext</code> records joined with <code>file_extents</code> keyed by <code>dstream_id</code> ; per-inode share is <code>dstream_dedup / dstream.refcnt</code> . Σ inode shares equals Σ <code>phys_ext.length · block size modulo</code> $\leq (\text{refcnt} - 1)$ bytes per clone-shared dstream (2-byte residue on the fixture). Productive discovery: <code>du -A</code> on macOS is apparent-size; no public per-file macOS oracle for clone-dedup exists. The crate ships a <code>real_size</code> field on <code>NamespaceEntry</code> .	validated-clone-dedup
EX-28	With root privileges, does the existing raw parser produce a self-consistent shape on a live <code>/dev/diskNsM</code> and agree with the fallback walker on the same mount? macOS returned <code>EPERM</code> on the first <code>read(2)</code> of <code>/dev/disk3s1</code> 's block zero even under <code>sudo</code> . The kernel's storage-system security policy refuses raw block access to the live boot data partition independently of POSIX permissions; SIP and the sealed-system-volume seal both contribute. Live boot raw mode is not viable on stock Apple silicon. Contribution: the public <code>apfs_fastindex::parity</code> comparator and the env-gated integration harness, both re-runnable on future hosts.	live-raw-blocked-by-kernel
EX-29	What user-visible local snapshots exist on the host, and what do reclaimable bytes look like? Zero user-visible TM snapshots; one sealed-system OS-update snapshot (SR-020-excluded). Reclaimable bytes have no public read-only macOS oracle (<code>tmutil thinlocalsnapshots</code> deletes the snapshot it measures), and EX-28's verdict blocks the raw-extent-diff alternative. Crate ships <code>snapshots::SnapshotVerdict</code> via <code>tmutil listlocalsnapshots + diskutil apfs listSnapshots</code> parsing.	blocked-no-user-snapshots
EX-30	Production-shape baseline for the GUI's <code>apfs-fastindex-scan</code> subprocess on <code>/Users/kai</code> (the largest single subtree the project owner routinely scans). Warm median 9.62 s, 162 k entries/s, 1.56 M entries, 172 k aggregates, 392 MB msgpack payload. The number anchors R4 cache acceptance: ≤ 1.0 s on unchanged-tree rescan, $\leq 5\%$ regression on cold-miss. The R4 v1 cache (<code>--cache</code> flag, ships this round) delivers 5–50× on stable trees and 1.0× on <code>/Users/kai</code> -class churn (the probe overhead exceeds the walk; mtime churn invalidates every signature).	production-baseline-recorded

Probe	Question and result	Verdict
EX-31	What’s the boot-root semantic the app should target? EX-28 foreclosed the raw-System-vs-raw-Data comparison. The only deliverable mode is Finder-visible startup root via POSIX traversal — which the existing fallback walker already produces, since POSIX traversal transparently follows firmlinks. The host’s <code>st_dev</code> reports identically for every path under / (the kernel hides the firmlink boundary from <code>stat</code>); the firmlink table at <code>/usr/share/firmlinks</code> is the authoritative source the walker already reads at scan-start.	api-only-already-correct
EX-32	Research scratchpad: what does the publicly-documented + reverse-engineered knowledge of APFS encryption cover, and what would a future feature require? Documentation of the keybag, per-file <code>crypto_state_t</code> , class keys, <code>INCOMPAT_ENC_ROLLED</code> , and the <code>wrapped_meta_crypto_state_t</code> of the volume superblock. Crate exposes <code>crypto_meta</code> as a non-production decoder module against synthetic fixtures. No decryption path is implemented; no feature follows without an oracle.	research-scratchpad
EX-33	Profile the <code>apfs_scan_search_names</code> FFI on the 1.56 M-entry <code>/Users/kai</code> baseline; ship one lift if there’s headroom. The pre-lift <code>Vec<String> + str::contains</code> ran 30–55 ms median per keystroke; the lifted flat- <code>Vec<u8>-buffer + memchr::memmem::Finder</code> runs 9–12 ms median for typical 4–12-character queries, $>12\times$ on zero-match queries (memory-bandwidth floor). Comfortably inside the “feels live” threshold.	validated-flat-buffer-memchr-lift